BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ TP. HỒ CHÍ MINH
KHOA CÔNG NGHỆ THÔNG TIN

[Chèn logo HUTECH]

ĐỒ ÁN MÔN HỌC
CƠ SỞ DỮ LIỆU NÂNG CAO

TÌM HIỂU PHÂN MẢNH HỖN HỢP TRONG CSDL PHÂN TÁN VÀ ỨNG DỤNG CHO QUẢN LÝ ĐƠN HÀNG

Ngành:	CÔNG NGHỆ THÔNG TIN	
Giảng viên hướng dẫn:	Nguyễn Văn Danh	
Sinh viên thực hiện:	Trần Gia Thịnh	MSSV: 2380602136
	Phạm Minh Triết	MSSV: 2380605829
Lớp:	……	

TP. Hồ Chí Minh, tháng 10 năm 2026

LỜI CẢM ƠN

Để hoàn thành đồ án môn học Cơ Sở Dữ Liệu Nâng Cao, nhóm chúng em xin gửi lời cảm ơn chân thành đến thầy Nguyễn Văn Danh, giảng viên hướng dẫn. Trong suốt quá trình học tập và thực hiện đề tài, thầy đã tận tình giảng dạy, định hướng và góp ý, giúp nhóm hiểu rõ hơn các vấn đề trong thiết kế cơ sở dữ liệu phân tán.

Nhóm cũng xin cảm ơn Khoa Công nghệ Thông tin, Trường Đại học Công nghệ TP. Hồ Chí Minh đã tạo điều kiện về môi trường học tập và tài liệu tham khảo.

Do thời gian và kiến thức còn hạn chế, đồ án khó tránh khỏi thiếu sót. Nhóm rất mong nhận được ý kiến đóng góp của thầy để hoàn thiện hơn.

Nhóm sinh viên thực hiện: Trần Gia Thịnh – Phạm Minh Triết

NHẬN XÉT CỦA GIẢNG VIÊN HƯỚNG DẪN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

TP. Hồ Chí Minh, ngày ..... tháng ..... năm 2026

Giảng viên hướng dẫn

BẢNG PHÂN CÔNG CÔNG VIỆC

Phân công công việc trong nhóm

STT	Họ và tên	MSSV	Nhiệm vụ	Mức độ hoàn thành
1	Trần Gia Thịnh	2380602136	Tìm hiểu tổng quan CSDL phân tán, phân mảnh ngang, phân mảnh dọc và phân mảnh hỗn hợp (Chương 1, 2, 3); dựng slide và thuyết trình phần lý thuyết	100%
2	Phạm Minh Triết	2380605829	Phân tích bài toán quản lý đơn hàng, thiết kế phân mảnh và cấp phát, cài đặt thực nghiệm trên SQL Server (Chương 4); dựng slide và thuyết trình phần ứng dụng	100%
	Cả nhóm		Lời mở đầu, kết luận (Chương 5), rà soát và hoàn thiện báo cáo	100%

MỤC LỤC

LỜI MỞ ĐẦU 1
Lý do chọn đề tài 1
Mục tiêu đề tài 1
Đối tượng và phạm vi 1
Phương pháp thực hiện 2
Bố cục báo cáo 2
CHƯƠNG 1. TỔNG QUAN VỀ CƠ SỞ DỮ LIỆU PHÂN TÁN 3
1.1 Khái niệm cơ sở dữ liệu phân tán 3
1.2 So sánh CSDL tập trung và CSDL phân tán 4
1.3 Kiến trúc tham chiếu của CSDL phân tán 4
1.4 Tính trong suốt trong CSDL phân tán 5
1.5 Thiết kế cơ sở dữ liệu phân tán 6
1.5.1 Hai hướng tiếp cận 6
1.5.2 Hai bài toán chính: phân mảnh và cấp phát 6
1.6 Kết chương 7
CHƯƠNG 2. PHÂN MẢNH DỮ LIỆU TRONG CSDL PHÂN TÁN 8
2.1 Lý do và đơn vị phân mảnh 8
2.2 Các quy tắc đúng đắn của phân mảnh 9
2.3 Phân mảnh ngang nguyên thủy 9
2.3.1 Định nghĩa 9
2.3.2 Vị từ đơn giản và vị từ hội sơ cấp 9
2.3.3 Tính đầy đủ và cực tiểu của tập vị từ 10
2.3.4 Ví dụ 10
2.4 Phân mảnh ngang dẫn xuất 11
2.4.1 Định nghĩa 11
2.4.2 Ví dụ 11
2.4.3 Tính đúng đắn và lựa chọn quan hệ chủ 12
2.5 Phân mảnh dọc 12

2.5.1 Định nghĩa 12
2.5.2 Ma trận sử dụng và ma trận ái lực 12
2.5.3 Thuật toán BEA 13
2.5.4 Thuật toán SPLIT 13
2.5.5 Ví dụ 13
2.6 Cấp phát và nhân bản 15
2.7 Kết chương 16
CHƯƠNG 3. PHÂN MẢNH HỖN HỢP 17
3.1 Định nghĩa và động cơ sử dụng 17
3.1.1 Động cơ 17
3.1.2 Định nghĩa 17
3.2 Chiến lược ngang trước, dọc sau (HV) 18
3.3 Chiến lược dọc trước, ngang sau (VH) 19
3.3.1 Quan hệ giữa HV và VH 20
3.4 Cây phân mảnh và biểu diễn bằng đại số quan hệ 20
3.5 Tái thiết quan hệ toàn cục 21
3.6 Kiểm tra tính đúng đắn 21
3.7 Cấp phát mảnh hỗn hợp và ảnh hưởng đến xử lý 22
3.7.1 Cấp phát 22
3.7.2 Ảnh hưởng đến xử lý truy vấn 22
3.7.3 Ảnh hưởng đến cập nhật và giao dịch 23
3.8 Ưu điểm, hạn chế và so sánh 23
3.9 Hỗ trợ trong các hệ quản trị thực tế 24
3.10 Kết chương 24
CHƯƠNG 4. ỨNG DỤNG PHÂN MẢNH HỖN HỢP CHO HỆ THỐNG QUẢN LÝ ĐƠN HÀNG 26
4.1 Mô tả bài toán 26
4.2 Lược đồ toàn cục 26
4.3 Phân tích ứng dụng 27
4.4 Thiết kế phân mảnh 28

4.4.1 KhachHang – phân mảnh hỗn hợp HV 29
4.4.2 NhanVien – phân mảnh hỗn hợp VH 29
4.4.3 DonHang, ChiTietDonHang – phân mảnh ngang dẫn xuất 30
4.4.4 ChiNhanh, SanPham – nhân bản toàn phần 30
4.5 Kiểm tra tính đúng đắn 30
4.6 Lược đồ cấp phát 31
4.7 Cài đặt thực nghiệm trên SQL Server 32
4.7.1 Môi trường và cách mô phỏng 32
4.7.2 Tạo các mảnh 32
4.7.3 Tái thiết và kiểm tra tính đúng đắn 33
4.7.4 Truy vấn minh họa 35
4.7.5 Giao dịch phân tán 36
4.8 Đánh giá 37
4.9 Kết chương 37
CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 39
5.1 Kết quả đạt được 39
5.2 Hạn chế 39
5.3 Hướng phát triển 39
TÀI LIỆU THAM KHẢO 41
PHỤ LỤC 42
Phụ lục A. Script tái thiết và kiểm tra tính đúng đắn 42
Phụ lục B. Script truy vấn minh họa và giao dịch phân tán 46

DANH MỤC TỪ VIẾT TẮT

Danh mục từ viết tắt

Từ viết tắt	Tiếng Anh	Tiếng Việt
CSDL	Database	Cơ sở dữ liệu
CSDLPT	Distributed Database	Cơ sở dữ liệu phân tán
DBMS	Database Management System	Hệ quản trị cơ sở dữ liệu
DDBMS	Distributed Database Management System	Hệ quản trị cơ sở dữ liệu phân tán
HV	Horizontal – Vertical	Phân mảnh hỗn hợp ngang trước, dọc sau
VH	Vertical – Horizontal	Phân mảnh hỗn hợp dọc trước, ngang sau
BEA	Bond Energy Algorithm	Thuật toán năng lượng liên kết
AA	Attribute Affinity matrix	Ma trận ái lực thuộc tính
CA	Clustered Affinity matrix	Ma trận ái lực đã gom cụm
PK	Primary Key	Khóa chính
2PC	Two-Phase Commit	Giao thức cam kết hai pha
MSDTC	Microsoft Distributed Transaction Coordinator	Bộ điều phối giao dịch phân tán của Microsoft
SSMS	SQL Server Management Studio	Công cụ quản trị SQL Server

DANH MỤC HÌNH

Hình 1.1: *Mô hình cơ sở dữ liệu phân tán gồm bốn site* 3

Hình 1.2: *Các mức lược đồ trong kiến trúc tham chiếu của CSDL phân tán* 5

Hình 2.1: *Minh họa ba kiểu phân mảnh* 8

Hình 2.2: *Chuỗi phân mảnh ngang dẫn xuất KhachHang – DonHang – ChiTietDonHang* 11

Hình 3.1: *Cây phân mảnh hỗn hợp HV của quan hệ KhachHang* 19

Hình 3.2: *Cây phân mảnh hỗn hợp VH của quan hệ NhanVien*.....20

Hình 4.1: *Lược đồ quan hệ toàn cục của hệ thống quản lý đơn hàng* 27

Hình 4.2: *Ma trận sử dụng thuộc tính của quan hệ KhachHang* 28

Hình 4.3: *Lược đồ cấp phát các mảnh cho bốn site* 31

DANH MỤC BẢNG

Bảng 1.1: *So sánh CSDL tập trung và CSDL phân tán* 4

Bảng 2.1: *Ma trận sử dụng thuộc tính use(q_i, A_j)* 14

Bảng 2.2: *Tần suất truy cập $acc_l(q_i)$ tại các site (lần/tháng)* 14

Bảng 2.3: *Ma trận ái lực AA* 14

Bảng 2.4: *Ma trận ái lực gom cụm CA* 15

Bảng 2.5: *Giá trị z tại các điểm cắt* 15

Bảng 3.1: *Nhu cầu truy cập KhachHang theo dòng và theo cột* 17

Bảng 3.2: *So sánh ba kiểu phân mảnh* 23

Bảng 4.1: *Các site trong hệ thống* 26

Bảng 4.2: *Các ứng dụng chính của hệ thống* 27

Bảng 4.3: *Kiểu phân mảnh của từng quan hệ* 28

Bảng 4.4: *Kiểm tra tính đúng đắn của phân mảnh hỗn hợp* 30

Bảng 4.5: *Lược đồ cấp phát* 31

Bảng 4.6: *Các script cài đặt* 32

Bảng 4.7: *Số bộ trong các mảnh sau khi phân mảnh* 33

Bảng 4.8: *Kết quả kiểm tra tính đúng đắn* 34

Bảng 4.9: *Báo cáo công nợ theo miền (Q3)* 35

Bảng 4.10: *Doanh thu theo chi nhánh, không tính đơn đã hủy (Q6)* 36

LỜI MỞ ĐẦU

Lý do chọn đề tài

Các doanh nghiệp hiện nay thường hoạt động trên nhiều địa điểm: trụ sở chính, chi nhánh ở nhiều tỉnh thành, kho hàng, cửa hàng. Nếu toàn bộ dữ liệu đặt tập trung tại một máy chủ, mọi giao dịch ở chi nhánh đều phải gửi về trung tâm. Điều đó làm tăng thời gian phản hồi và tải trên đường truyền, và cả hệ thống ngừng hoạt động khi máy chủ trung tâm gặp sự cố. Cơ sở dữ liệu phân tán giải quyết vấn đề này bằng cách đặt dữ liệu tại nhiều site, gần nơi dữ liệu được sử dụng, nhưng vẫn cho người dùng thấy như một cơ sở dữ liệu thống nhất.

Vấn đề cốt lõi khi thiết kế cơ sở dữ liệu phân tán là **phân mảnh**: chia mỗi quan hệ thành những phần nhỏ hơn để đặt ở các site khác nhau. Phân mảnh ngang chia theo dòng, phân mảnh dọc chia theo cột. Tuy nhiên, trong thực tế, các nhóm người dùng thường vừa chỉ quan tâm đến một số dòng, vừa chỉ quan tâm đến một số cột. Khi đó, chỉ dùng một kiểu phân mảnh là không đủ, và **phân mảnh hỗn hợp**, tức là kết hợp cả hai kiểu, là lựa chọn phù hợp.

Bài toán quản lý đơn hàng của một công ty có nhiều chi nhánh là ví dụ điển hình. Chi nhánh chỉ làm việc với khách hàng và đơn hàng của mình. Phòng kế toán ở trụ sở cần thông tin công nợ của khách hàng ở mọi chi nhánh. Phòng nhân sự quản lý dữ liệu lương nhạy cảm, không được để lộ ra chi nhánh. Vì vậy nhóm chọn đề tài **"Tìm hiểu Phân mảnh hỗn hợp trong CSDL Phân tán và ứng dụng cho quản lý đơn hàng"** để vừa nắm vững lý thuyết, vừa thấy được cách áp dụng vào một bài toán cụ thể.

Mục tiêu đề tài

- Trình bày các khái niệm cơ bản về cơ sở dữ liệu phân tán và bài toán thiết kế phân tán.
- Tìm hiểu các kiểu phân mảnh ngang, dọc và các quy tắc đúng đắn của phân mảnh.
- Tìm hiểu sâu về phân mảnh hỗn hợp: định nghĩa, hai chiến lược HV và VH, cây phân mảnh, cách tái thiết và kiểm tra tính đúng đắn.
- Thiết kế phân mảnh hỗn hợp và cấp phát cho hệ thống quản lý đơn hàng, sau đó cài đặt thực nghiệm trên SQL Server.

Đối tượng và phạm vi

- **Đối tượng:** kỹ thuật phân mảnh dữ liệu trong cơ sở dữ liệu phân tán quan hệ, trọng tâm là phân mảnh hỗn hợp.

- **Phạm vi:** bài toán quản lý đơn hàng của một công ty giả định có ba chi nhánh và một trụ sở chính. Thực nghiệm mô phỏng mỗi site bằng một cơ sở dữ liệu trên Microsoft SQL Server. Các vấn đề tối ưu truy vấn phân tán, điều khiển đồng thời và phục hồi chỉ được đề cập ở mức liên quan đến thiết kế.

Phương pháp thực hiện

- Nghiên cứu tài liệu: giáo trình Principles of Distributed Database Systems của Özsu và Valduriez, bài giảng môn học và tài liệu kỹ thuật của Microsoft.
- Phân tích – thiết kế: xác định các ứng dụng chính, lập ma trận sử dụng thuộc tính, thiết kế phân mảnh và cấp phát, chứng minh tính đúng đắn.
- Thực nghiệm: cài đặt các site, phân mảnh dữ liệu, tạo view tái thiết và viết truy vấn kiểm tra tự động.

Bố cục báo cáo

Ngoài phần mở đầu và kết luận, báo cáo gồm các chương:

- **Chương 1 – Tổng quan về cơ sở dữ liệu phân tán:** khái niệm, kiến trúc, tính trong suốt và các vấn đề thiết kế.
- **Chương 2 – Phân mảnh dữ liệu trong CSDL phân tán:** các quy tắc đúng đắn, phân mảnh ngang nguyên thủy, ngang dẫn xuất và phân mảnh dọc.
- **Chương 3 – Phân mảnh hỗn hợp:** định nghĩa, chiến lược HV và VH, cây phân mảnh, tái thiết, tính đúng đắn và cấp phát.
- **Chương 4 – Ứng dụng phân mảnh hỗn hợp cho hệ thống quản lý đơn hàng:** phân tích bài toán, thiết kế, cài đặt và đánh giá.
- **Chương 5 – Kết luận và hướng phát triển.**

CHƯƠNG 1. TỔNG QUAN VỀ CƠ SỞ DỮ LIỆU PHÂN TÁN

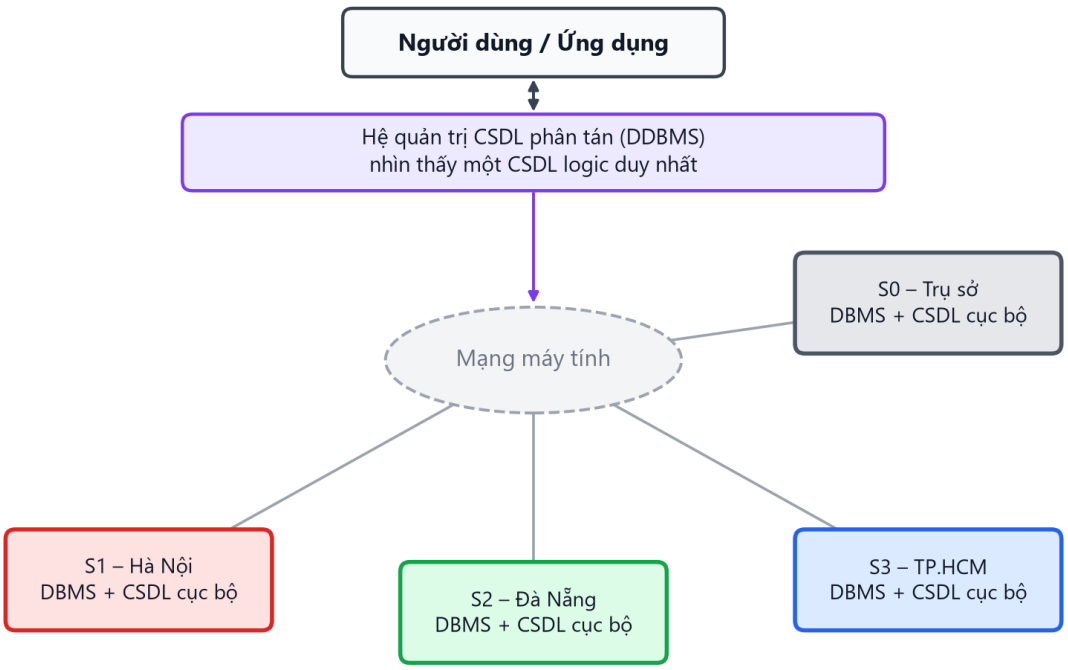
1.1 Khái niệm cơ sở dữ liệu phân tán

Theo Özsu và Valduriez [1], **cơ sở dữ liệu phân tán** (CSDLPT) là một tập hợp nhiều cơ sở dữ liệu có liên quan với nhau về mặt logic, được phân bố trên các nút của một mạng máy tính. **Hệ quản trị cơ sở dữ liệu phân tán** (DDBMS) là phần mềm quản lý CSDLPT, đồng thời làm cho sự phân tán trở nên **trong suốt** đối với người dùng.

Từ định nghĩa trên, có thể rút ra hai đặc điểm quan trọng:

- **Phân tán về vật lý:** dữ liệu không nằm trên một máy mà được lưu tại nhiều *site*. Mỗi site có hệ quản trị CSDL cục bộ, có khả năng xử lý độc lập, và các site trao đổi dữ liệu qua mạng.
- **Thống nhất về logic:** dữ liệu ở các site không rời rạc mà thuộc về cùng một lược đồ toàn cục. Một tập hợp file đặt rải rác trên nhiều máy, hay nhiều CSDL độc lập không liên quan đến nhau, đều không phải là CSDLPT.

Hình 1.1 minh họa một CSDLPT gồm bốn site. Người dùng làm việc với DDBMS như với một CSDL duy nhất; DDBMS chịu trách nhiệm xác định dữ liệu nằm ở site nào và phối hợp các site để trả lời truy vấn.



Hình 1.1: *Mô hình cơ sở dữ liệu phân tán gồm bốn site*

1.2 So sánh CSDL tập trung và CSDL phân tán

Trong mô hình tập trung, toàn bộ dữ liệu đặt tại một máy chủ, các máy khách ở xa truy cập qua mạng. Mô hình này đơn giản nhưng có những hạn chế rõ ràng khi tổ chức trải rộng về địa lý. Bảng 1.1 so sánh hai mô hình theo các tiêu chí chính.

Bảng 1.1: *So sánh CSDL tập trung và CSDL phân tán*

Tiêu chí	CSDL tập trung	CSDL phân tán
Vị trí dữ liệu	Một máy chủ duy nhất	Nhiều site nối mạng
Truy cập từ chi nhánh	Luôn phải truy cập từ xa	Phần lớn truy cập cục bộ
Độ sẵn sàng	Máy chủ hỏng thì cả hệ thống dừng	Một site hỏng, các site khác vẫn hoạt động
Khả năng mở rộng	Nâng cấp máy chủ (mở rộng theo chiều dọc)	Thêm site mới (mở rộng theo chiều ngang)
Độ phức tạp	Thấp	Cao: thiết kế phân tán, truy vấn và giao dịch phân tán
Kiểm soát và bảo mật	Tập trung, dễ quản lý	Phải bảo vệ dữ liệu ở nhiều nơi và trên đường truyền

Ưu điểm của CSDLPT gồm:

- **Dữ liệu đặt gần nơi sử dụng:** giảm thời gian phản hồi và lưu lượng mạng, vì đa số truy vấn được trả lời bằng dữ liệu cục bộ.
- **Tăng độ tin cậy và độ sẵn sàng:** khi dữ liệu được nhân bản, sự cố ở một site không làm gián đoạn toàn hệ thống.
- **Xử lý song song:** một truy vấn lớn có thể được tách thành nhiều truy vấn con chạy đồng thời trên các site.
- **Phù hợp với cơ cấu tổ chức:** mỗi đơn vị quản lý dữ liệu của mình nhưng vẫn chia sẻ được với toàn tổ chức.
- **Dễ mở rộng:** khi mở thêm chi nhánh chỉ cần bổ sung site mới.

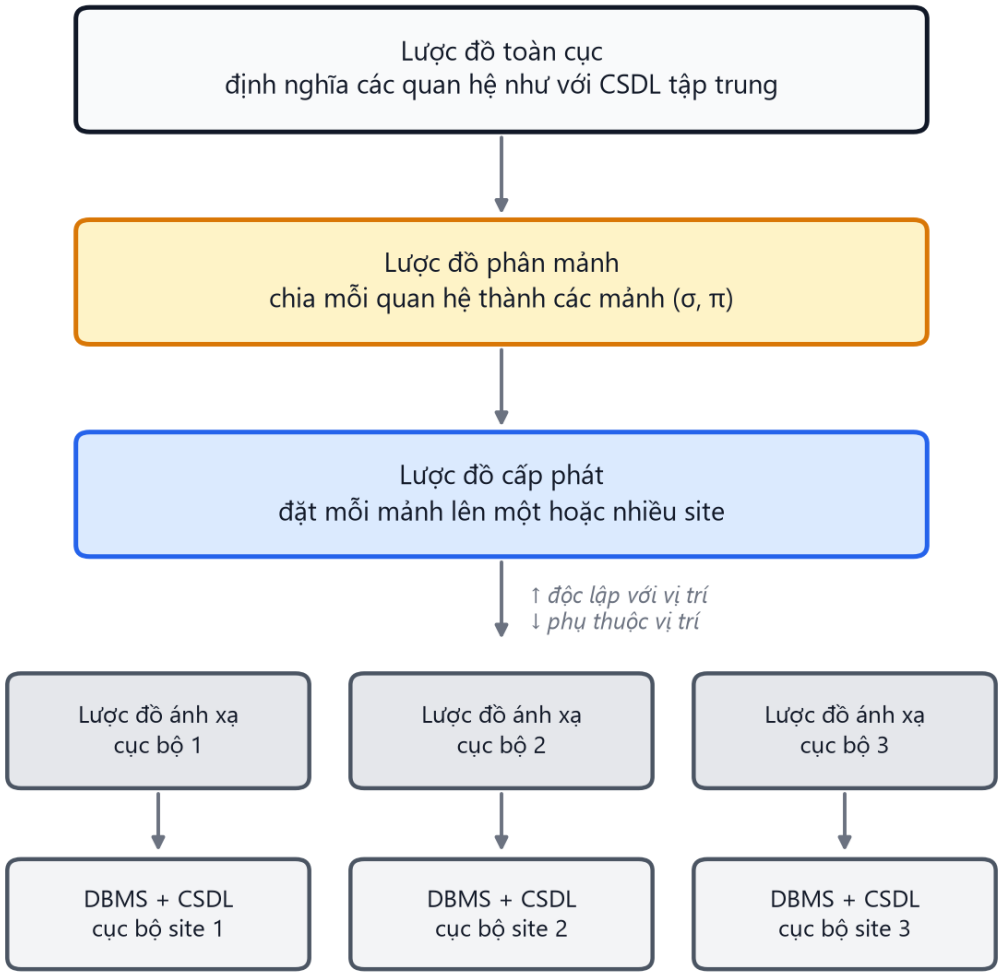
Bên cạnh đó, CSDLPT cũng có những **nhược điểm**: thiết kế phức tạp hơn; xử lý truy vấn và giao dịch phải tính đến chi phí truyền thông; việc đảm bảo nhất quán giữa các bản nhân bản và thực hiện giao dịch trên nhiều site đòi hỏi các giao thức như cam kết hai pha (2PC); chi phí quản trị cao hơn.

1.3 Kiến trúc tham chiếu của CSDL phân tán

Kiến trúc tham chiếu của CSDLPT [1], [2] mô tả dữ liệu qua nhiều mức lược đồ (Hình 1.2):

- **Lược đồ toàn cục:** định nghĩa toàn bộ dữ liệu của hệ thống dưới dạng các quan hệ, giống như một CSDL tập trung. Người dùng và ứng dụng làm việc ở mức này.
- **Lược đồ phân mảnh:** mô tả cách mỗi quan hệ toàn cục được chia thành các mảnh, bằng các phép chọn (σ) và phép chiếu (π) của đại số quan hệ.
- **Lược đồ cấp phát:** xác định mỗi mảnh được đặt ở site nào. Một mảnh có thể được đặt ở một site (không nhân bản) hoặc ở nhiều site (nhân bản).
- **Lược đồ ánh xạ cục bộ:** ánh xạ các mảnh vật lý tại mỗi site sang các đối tượng của DBMS cục bộ, chẳng hạn bảng của SQL Server hoặc Oracle.

Ba mức trên cùng không phụ thuộc vào vị trí và hệ quản trị cụ thể; mức ánh xạ cục bộ phụ thuộc vào từng site. Sự tách biệt này là cơ sở để đạt được tính trong suốt.



Hình 1.2: *Các mức lược đồ trong kiến trúc tham chiếu của CSDL phân tán*

1.4 Tính trong suốt trong CSDL phân tán

Tính trong suốt nghĩa là người dùng không cần biết các chi tiết cài đặt của hệ thống phân tán. Có thể phân biệt các mức trong suốt sau, theo thứ tự từ cao xuống thấp:

- **Trong suốt phân mảnh:** người dùng truy vấn trên quan hệ toàn cục mà không cần biết quan hệ đó đã bị chia thành những mảnh nào. Đây là mức trong suốt cao nhất.
- **Trong suốt vị trí:** người dùng biết các mảnh nhưng không cần biết mỗi mảnh nằm ở site nào.
- **Trong suốt nhân bản:** người dùng không cần biết một mảnh có bao nhiêu bản sao, cũng không phải tự cập nhật từng bản sao.
- **Trong suốt ánh xạ cục bộ:** người dùng không cần biết mỗi site dùng hệ quản trị nào và lưu dữ liệu ra sao.

Mức trong suốt càng cao thì việc viết ứng dụng càng đơn giản, nhưng DDBMS phải đảm nhận nhiều việc hơn, ví dụ phải tự phân rã một truy vấn toàn cục thành các truy vấn trên từng mảnh. Trong thực nghiệm ở Chương 4, nhóm đạt được trong suốt phân mảnh và trong suốt vị trí bằng view và synonym của SQL Server.

1.5 Thiết kế cơ sở dữ liệu phân tán

1.5.1 Hai hướng tiếp cận

- **Thiết kế từ trên xuống (top-down):** xuất phát từ yêu cầu của hệ thống, xây dựng lược đồ toàn cục, sau đó thiết kế phân mảnh và cấp phát, cuối cùng là lược đồ cục bộ tại mỗi site. Hướng này phù hợp khi xây dựng hệ thống mới, và cũng là hướng đề tài áp dụng.
- **Thiết kế từ dưới lên (bottom-up):** khi đã có sẵn nhiều CSDL độc lập, cần tích hợp chúng thành một lược đồ toàn cục chung. Hướng này thường gặp khi hợp nhất các hệ thống có sẵn.

1.5.2 Hai bài toán chính: phân mảnh và cấp phát

Trong thiết kế từ trên xuống, sau khi có lược đồ toàn cục, người thiết kế phải giải quyết hai bài toán:

- **Phân mảnh:** chia mỗi quan hệ thành các mảnh sao cho mỗi mảnh chứa đúng phần dữ liệu mà một nhóm ứng dụng cần. Đây là nội dung chính của Chương 2 và Chương 3.
- **Cấp phát:** đặt các mảnh lên các site sao cho tối thiểu hóa chi phí truy cập và truyền thông, đồng thời đáp ứng yêu cầu về độ sẵn sàng. Có ba phương án: không nhân bản (mỗi mảnh ở đúng một site), nhân bản một phần, và nhân bản toàn phần.

Cả hai bài toán đều cần thông tin đầu vào về dữ liệu và ứng dụng: các truy vấn chính, thuộc tính và vị từ mà chúng sử dụng, nơi phát sinh, tần suất thực hiện. Phân

mảnh tốt mà cấp phát không hợp lý thì vẫn phát sinh nhiều truy cập từ xa, nên hai bài toán phải được xem xét cùng nhau.

1.6 Kết chương

Chương 1 đã trình bày khái niệm, ưu nhược điểm, kiến trúc tham chiếu và các mức trong suốt của CSDL phân tán, cùng hai bài toán thiết kế là phân mảnh và cấp phát. Chương 2 sẽ đi vào chi tiết bài toán phân mảnh với hai kiểu cơ bản là phân mảnh ngang và phân mảnh dọc.

CHƯƠNG 2. PHÂN MẢNH DỮ LIỆU TRONG CSDL PHÂN TÁN

2.1 Lý do và đơn vị phân mảnh

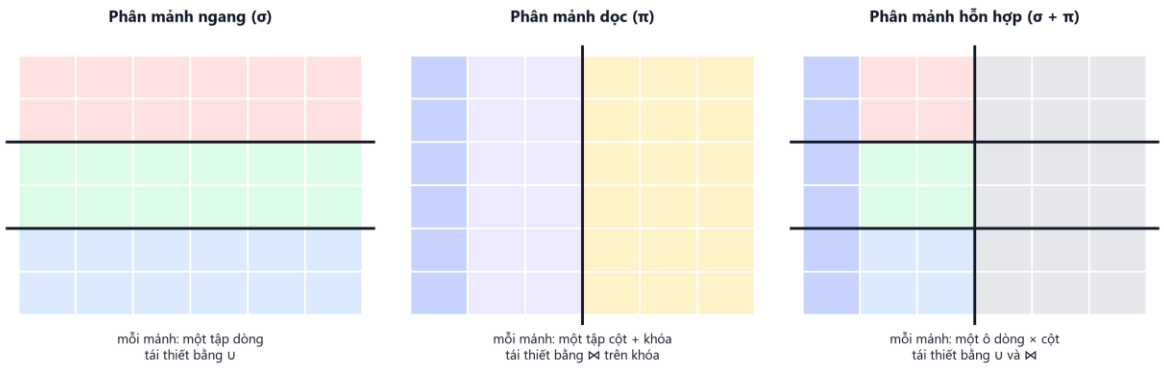
Nếu đơn vị phân tán là cả một quan hệ, sẽ có hai lựa chọn đều không hiệu quả. Đặt quan hệ ở một site thì các site khác phải truy cập từ xa. Nhân bản quan hệ ra mọi site thì tốn không gian lưu trữ, và mỗi lần cập nhật phải sửa tất cả các bản sao. Trong khi đó, mỗi ứng dụng thường chỉ dùng một **tập con** của quan hệ, ví dụ chi nhánh Hà Nội chỉ cần khách hàng ở Hà Nội. Vì vậy, đơn vị phân tán hợp lý là các **mảnh** (fragment) của quan hệ.

Phân mảnh mang lại các lợi ích:

- **Tăng tính cục bộ:** mỗi mảnh đặt ở nơi sử dụng nó nhiều nhất, phần lớn truy vấn được trả lời bằng dữ liệu tại chỗ.
- **Xử lý song song:** một truy vấn trên quan hệ có thể tách thành nhiều truy vấn con chạy đồng thời trên các mảnh.
- **Bảo mật:** dữ liệu nhạy cảm được tách riêng và chỉ đặt ở những site có quyền truy cập.

Phân mảnh cũng có cái giá phải trả. Những ứng dụng cần dữ liệu ở nhiều mảnh phải ghép các mảnh lại, phát sinh chi phí truyền thông và xử lý. Việc kiểm tra ràng buộc toàn vẹn liên quan đến nhiều mảnh cũng phức tạp hơn. Do đó **mức độ phân mảnh** phải vừa phải: không phân mảnh thì mất tính cục bộ, phân mảnh quá nhỏ (đến từng bộ hoặc từng thuộc tính) thì chi phí ghép lại rất lớn. Mức độ hợp lý được xác định dựa trên các ứng dụng sử dụng CSDL.

Có ba kiểu phân mảnh cơ bản (Hình 2.1): **phân mảnh ngang** (theo dòng), **phân mảnh dọc** (theo cột) và **phân mảnh hỗn hợp** (kết hợp cả hai).



Hình 2.1: *Minh họa ba kiểu phân mảnh*

2.2 Các quy tắc đúng đắn của phân mảnh

Giả sử quan hệ R được phân mảnh thành các mảnh R_1, R_2, …, R_n. Phân mảnh được gọi là đúng đắn nếu thỏa ba quy tắc [1]:

- **Tính đầy đủ (completeness):** mỗi mục dữ liệu của R (một bộ đối với phân mảnh ngang, một thuộc tính đối với phân mảnh dọc) đều có mặt trong ít nhất một mảnh R_i. Quy tắc này đảm bảo phân mảnh không làm mất dữ liệu.
- **Tính tái thiết (reconstruction):** tồn tại một phép toán quan hệ ∇ sao cho $R = \nabla R_i$. Với phân mảnh ngang, ∇ là phép hợp ($\cup$); với phân mảnh dọc, ∇ là phép kết nối ($\bowtie$) trên khóa. Quy tắc này đảm bảo các ràng buộc phụ thuộc giữa dữ liệu được bảo toàn.
- **Tính rời nhau (disjointness):** với phân mảnh ngang, một bộ thuộc R_j thì không thuộc R_k ($j \neq k$). Với phân mảnh dọc, các mảnh chỉ được trùng nhau ở khóa chính; khóa phải được lặp lại ở mọi mảnh để có thể tái thiết.

2.3 Phân mảnh ngang nguyên thủy

2.3.1 Định nghĩa

Phân mảnh ngang chia quan hệ theo các bộ. Mỗi mảnh là kết quả của một phép chọn trên quan hệ:

$$R_i = \sigma_{F_i}(R), \quad i = 1..n$$

trong đó F_i là vị từ chọn của mảnh R_i. Gọi là phân mảnh **nguyên thủy** vì vị từ được định nghĩa trên chính các thuộc tính của R.

2.3.2 Vị từ đơn giản và vị từ hội sơ cấp

- **Vị từ đơn giản** có dạng p_j: $A_i\ \theta$ Giá_trị, với A_i là thuộc tính của R và $\theta \in \{=, <, \leq, >, \geq, \neq\}$. Ví dụ: p_1: MaCN = 'CN01'.
- **Vị từ hội sơ cấp (minterm)** trên tập vị từ đơn giản $Pr = \{p_1, \ldots, p_m\}$ là phép hội của tất cả các vị từ trong Pr, mỗi vị từ xuất hiện ở dạng khẳng định hoặc phủ định:

$$m_k = p_1^* \wedge p_2^* \wedge \ldots \wedge p_m^*, \quad \text{với } p_i^* \in \{p_i, \neg p_i\}$$

Mỗi minterm xác định một mảnh ngang, gọi là **mảnh hội sơ cấp**. Với m vị từ sẽ có 2^m minterm, nhưng nhiều minterm mâu thuẫn và bị loại bỏ dựa trên ngữ nghĩa của thuộc tính.

Thông tin cần thu thập để thiết kế gồm: các vị từ đơn giản mà ứng dụng dùng trong điều kiện truy vấn; độ chọn của từng minterm (tỉ lệ bộ thỏa minterm); và tần suất truy cập của ứng dụng.

2.3.3 Tính đầy đủ và cực tiểu của tập vị từ

- Tập vị từ Pr là **đầy đủ** nếu hai bộ bất kỳ thuộc cùng một mảnh hội sơ cấp luôn có cùng xác suất được truy cập bởi mọi ứng dụng. Nói cách khác, mỗi mảnh phải đồng nhất về cách được sử dụng.
- Một vị từ là **có liên quan** (relevant) nếu nó tách một mảnh f thành hai mảnh f_1, f_2 được ít nhất một ứng dụng truy cập khác nhau. Tập Pr là **cực tiểu** nếu mọi vị từ trong đó đều có liên quan.

Thuật toán COM_MIN [1] sinh ra tập vị từ đầy đủ và cực tiểu Pr' từ tập vị từ ứng viên Pr:

1. Chọn một vị từ $p_i \in$ Pr chia quan hệ R thành hai phần được ít nhất một ứng dụng truy cập khác nhau; đưa p_i vào Pr'.
2. Lần lượt xét các vị từ còn lại. Vị từ nào chia một mảnh hiện có thành hai phần được truy cập khác nhau thì đưa vào Pr'.
3. Khi thêm vị từ mới, loại khỏi Pr' những vị từ đã trở nên không còn liên quan.
4. Lặp lại cho đến khi Pr' đầy đủ.

Thuật toán PHORIZONTAL dùng kết quả của COM_MIN: tính Pr' = COM_MIN(R, Pr), xác định tập minterm M trên Pr', rồi dùng các phép kéo theo (implication) giữa các vị từ để loại bỏ những minterm mâu thuẫn. Mỗi minterm còn lại cho một mảnh ngang.

2.3.4 Ví dụ

Xét quan hệ KhachHang(MaKH, HoTen, SDT, DiaChi, MaCN, HanMucNo, CongNo), với MaCN ∈ {'CN01', 'CN02', 'CN03'}. Ứng dụng tra cứu khách hàng ở mỗi chi nhánh chỉ truy cập khách thuộc chi nhánh đó, nên tập vị từ ứng viên là:

$$p_1: \text{MaCN} = \text{'CN01'} \quad p_2: \text{MaCN} = \text{'CN02'} \quad p_3: \text{MaCN} = \text{'CN03'}$$

Có $2^3 = 8$ minterm. Vì MaCN chỉ nhận một giá trị nên có các phép kéo theo $p_1 \Rightarrow \neg p_2 \wedge \neg p_3$ (tương tự cho p_2, p_3). Mọi minterm chứa từ hai vị từ khẳng định trở lên đều mâu thuẫn. Minterm $\neg p_1 \wedge \neg p_2 \wedge \neg p_3$ cũng mâu thuẫn vì miền giá trị của MaCN chỉ gồm ba giá trị trên. Còn lại ba minterm, rút gọn được thành p_1, p_2, p_3, cho ba mảnh:

$$\text{KH}_i = \sigma_{\text{MaCN}=\text{'CN0}i\text{'}}(\text{KhachHang}), \quad i = 1, 2, 3$$

Giả sử có thêm vị từ ứng viên p_4: HanMucNo ≥ 200.000.000. Nếu có một ứng dụng truy cập riêng nhóm khách có hạn mức lớn thường xuyên hơn, p_4 là vị từ có liên quan và mỗi mảnh trên sẽ tách đôi, tổng cộng sáu mảnh. Ngược lại, nếu mọi ứng dụng truy cập khách hàng như nhau bất kể hạn mức, p_4 không có liên quan và COM_MIN sẽ không đưa nó vào Pr'. Bài toán ở Chương 4 thuộc trường hợp thứ hai.

Ba mảnh KH_1, KH_2, KH_3 thỏa các quy tắc đúng đắn: **đầy đủ** vì $p_1 \vee p_2 \vee p_3$ luôn đúng trên miền giá trị của MaCN; **rời nhau** vì các vị từ loại trừ lẫn nhau; **tái thiết** được bằng KhachHang = $KH_1 \cup KH_2 \cup KH_3$.

2.4 Phân mảnh ngang dẫn xuất

2.4.1 Định nghĩa

Phân mảnh ngang dẫn xuất áp dụng cho một quan hệ **thành viên** (member), dựa trên phân mảnh của quan hệ **chủ** (owner) mà nó tham chiếu tới. Quan hệ chủ và thành viên được nối với nhau bằng một liên kết, thường là khóa ngoại. Nếu S là quan hệ chủ đã được phân mảnh thành S_1, …, S_n, thì quan hệ thành viên R được phân mảnh bằng phép nửa kết nối:

$$R_i = R \ltimes S_i, \quad i = 1..n$$

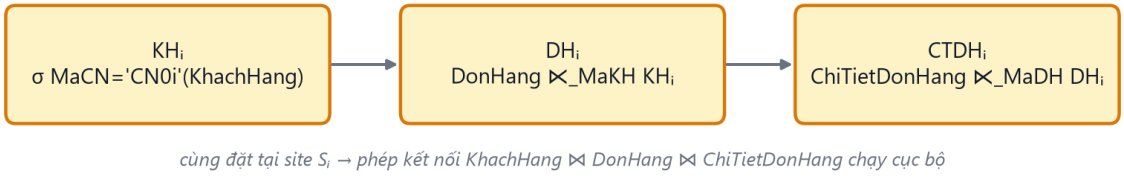
Phép nửa kết nối $R \ltimes S_i$ giữ lại những bộ của R kết nối được với ít nhất một bộ của S_i, và chỉ giữ các thuộc tính của R.

2.4.2 Ví dụ

Quan hệ DonHang(MaDH, NgayDat, MaKH, MaNV, TrangThai, TongTien) tham chiếu KhachHang qua MaKH. Để đơn hàng nằm cùng site với khách hàng, DonHang được phân mảnh dẫn xuất theo các mảnh KH_i ở mục 2.3.4:

$$DH_i = DonHang \ltimes_{MaKH} KH_i, \quad i = 1, 2, 3$$

Tương tự, ChiTietDonHang được phân mảnh dẫn xuất theo DonHang: $CTDH_i$ = ChiTietDonHang $\ltimes_{MaDH}$ DH_i. Kết quả là một chuỗi phân mảnh dẫn xuất (Hình 2.2).



Hình 2.2: *Chuỗi phân mảnh ngang dẫn xuất KhachHang – DonHang – ChiTietDonHang*

Lợi ích quan trọng nhất: các mảnh KH_i, DH_i, $CTDH_i$ cùng đặt tại một site, nên phép kết nối KhachHang ⋈ DonHang ⋈ ChiTietDonHang được thực hiện **cục bộ** tại từng site rồi hợp kết quả lại, không cần truyền dữ liệu giữa các site. Nhờ vậy đồ thị kết nối giữa các mảnh trở nên **đơn giản**: mỗi mảnh của quan hệ thành viên chỉ kết nối với đúng một mảnh của quan hệ chủ.

2.4.3 Tính đúng đắn và lựa chọn quan hệ chủ

- **Đầy đủ:** cần ràng buộc toàn vẹn tham chiếu. Nếu một đơn hàng có MaKH không tồn tại trong KhachHang, đơn đó sẽ không thuộc mảnh nào.
- **Rời nhau:** được đảm bảo khi mỗi bộ của quan hệ thành viên chỉ kết nối với bộ của đúng một mảnh chủ. Ở đây mỗi đơn hàng có đúng một MaKH, và mỗi khách hàng thuộc đúng một chi nhánh.
- **Tái thiết:** DonHang = $DH_1 \cup DH_2 \cup DH_3$.

Khi quan hệ thành viên có nhiều quan hệ chủ (DonHang tham chiếu cả KhachHang và NhanVien), ta chọn phân mảnh nào được nhiều ứng dụng sử dụng hơn, hoặc cho đặc tính kết nối tốt hơn. Trong bài toán của đề tài, nhân viên lập đơn luôn thuộc cùng chi nhánh với khách hàng, nên hai cách chọn cho cùng kết quả; nhóm chọn KhachHang vì ứng dụng lập đơn kết nối DonHang với KhachHang thường xuyên nhất.

2.5 Phân mảnh dọc

2.5.1 Định nghĩa

Phân mảnh dọc chia quan hệ theo các thuộc tính. Mỗi mảnh là phép chiếu của R lên một tập thuộc tính, và **mọi mảnh đều chứa khóa chính** K của R:

$$R_i = \pi_{K \cup A_i}(R), \quad i = 1..n$$

Tái thiết bằng phép kết nối trên khóa: $R = R_1 \bowtie_K R_2 \bowtie_K \dots \bowtie_K R_n$. Mục tiêu của phân mảnh dọc là gom vào cùng một mảnh những thuộc tính thường được truy cập cùng nhau, để mỗi ứng dụng chỉ cần đọc một mảnh.

Bài toán phân mảnh dọc phức tạp hơn phân mảnh ngang: với m thuộc tính không khóa, số cách chia là số Bell B(m), tăng rất nhanh (B(10) ≈ 115.000). Vì vậy người ta dùng các phương pháp heuristic, phổ biến nhất là thuật toán năng lượng liên kết BEA kết hợp với thuật toán SPLIT.

2.5.2 Ma trận sử dụng và ma trận ái lực

Cho tập ứng dụng $Q = \{q_1, \dots, q_k\}$ và các thuộc tính $A_1, \dots, A_m$:

- **Giá trị sử dụng:** use(q_i, A_j) = 1 nếu q_i dùng A_j, ngược lại bằng 0.

- **Tần suất truy cập:** $acc_l(q_i)$ là số lần q_i được thực hiện tại site S_l trong một khoảng thời gian.
- **Độ ái lực** giữa hai thuộc tính A_i, A_j đo mức độ chúng được truy cập cùng nhau:

$$aff(A_i, A_j) = \Sigma_{k:\, use(q_k,A_i)=1 \wedge use(q_k,A_j)=1} \; \Sigma_l \, acc_l(q_k)$$

Các giá trị aff tạo thành **ma trận ái lực AA** kích thước m × m.

2.5.3 Thuật toán BEA

Thuật toán năng lượng liên kết (Bond Energy Algorithm) do McCormick và cộng sự đề xuất [4], được Navathe và cộng sự áp dụng cho phân mảnh dọc [5], hoán vị các cột và dòng của AA để các thuộc tính có ái lực cao đứng cạnh nhau, tạo thành ma trận ái lực gom cụm CA. Độ liên kết giữa hai thuộc tính được định nghĩa:

$$bond(A_x, A_y) = \Sigma_{z=1..m} \, aff(A_z, A_x) \times aff(A_z, A_y)$$

Thuật toán đặt hai cột đầu tiên, sau đó lần lượt chèn từng cột A_k còn lại vào vị trí (giữa A_i và A_j) làm cực đại **đóng góp**:

$$cont(A_i, A_k, A_j) = 2 \cdot bond(A_i, A_k) + 2 \cdot bond(A_k, A_j) - 2 \cdot bond(A_i, A_j)$$

với quy ước bond với cột ở ngoài biên bằng 0. Khi các cột đã được sắp, các dòng được sắp theo cùng thứ tự.

2.5.4 Thuật toán SPLIT

Trên ma trận CA, chọn một điểm cắt chia các thuộc tính thành nhóm trên TA và nhóm dưới BA. Gọi TQ là tập ứng dụng chỉ dùng thuộc tính trong TA, BQ là tập ứng dụng chỉ dùng thuộc tính trong BA, OQ là các ứng dụng dùng cả hai nhóm; CTQ, CBQ, COQ là tổng tần suất truy cập tương ứng. Điểm cắt tốt nhất là điểm làm cực đại:

$$z = CTQ \times CBQ - COQ^2$$

Giá trị z lớn nghĩa là nhiều truy cập chỉ cần một mảnh và ít truy cập phải dùng cả hai mảnh.

2.5.5 Ví dụ

Xét các thuộc tính không khóa của KhachHang: A_1 = HoTen, A_2 = SDT, A_3 = DiaChi, A_4 = HanMucNo, A_5 = CongNo (khóa MaKH được lặp lại ở mọi mảnh nên không tham gia gom cụm). Có bốn ứng dụng: q_1 tra cứu khách hàng, q_2 in phiếu giao hàng, q_3 kiểm tra hạn mức, q_4 báo cáo công nợ. Ma trận sử dụng và tần suất truy cập mỗi tháng tại các site cho ở Bảng 2.1 và Bảng 2.2.

Bảng 2.1: *Ma trận sử dụng thuộc tính use(q_i, A_j)*

	HoTen	**SDT**	**DiaChi**	**HanMucNo**	**CongNo**
q_1	1	1	1	0	0
q_2	1	0	1	0	0
q_3	0	0	0	1	1
q_4	0	0	0	0	1

Bảng 2.2: *Tần suất truy cập $acc_l(q_i)$ tại các site (lần/tháng)*

	S0 Trụ sở	**S1 Hà Nội**	**S2 Đà Nẵng**	**S3 TP.HCM**	**Tổng**
q_1	0	300	200	400	900
q_2	0	150	100	200	450
q_3	80	0	0	0	80
q_4	30	0	0	0	30

Ví dụ, aff(HoTen, DiaChi) = 900 + 450 = 1350 vì cả q_1 và q_2 đều dùng hai thuộc tính này; aff(HanMucNo, CongNo) = 80 vì chỉ q_3 dùng cả hai. Ma trận ái lực đầy đủ ở Bảng 2.3.

Bảng 2.3: *Ma trận ái lực AA*

	HoTen	**SDT**	**DiaChi**	**HanMucNo**	**CongNo**
HoTen	1350	900	1350	0	0
SDT	900	900	900	0	0
DiaChi	1350	900	1350	0	0
HanMucNo	0	0	0	80	80
CongNo	0	0	0	80	110

Áp dụng BEA: đặt trước HoTen, SDT. Chèn DiaChi: cont tại vị trí đầu (trước HoTen) bằng 2·bond(DiaChi, HoTen) = 2 × (1350·1350 + 900·900 + 1350·1350) = 8.910.000, bằng với vị trí giữa HoTen và SDT và lớn hơn vị trí cuối (6.480.000); chọn vị trí đầu, được thứ tự DiaChi, HoTen, SDT. Chèn HanMucNo: vị trí đầu và cuối cho cont = 0, các vị trí giữa cho giá trị âm; chọn vị trí đầu. Chèn CongNo: vị trí đầu và vị trí giữa HanMucNo – DiaChi cùng cho cont = 30.400, lớn nhất; chọn vị trí đầu (khi bằng nhau, chọn vị trí xuất hiện trước). Thứ tự cuối cùng là CongNo, HanMucNo, DiaChi, HoTen, SDT, cho ma trận CA ở Bảng 2.4. Có thể thấy hai khối ái lực cao tách biệt rõ trên đường chéo.

Bảng 2.4: *Ma trận ái lực gom cụm CA*

	CongNo	**HanMucNo**	**DiaChi**	**HoTen**	**SDT**
CongNo	110	80	0	0	0
HanMucNo	80	80	0	0	0
DiaChi	0	0	1350	1350	900
HoTen	0	0	1350	1350	900
SDT	0	0	900	900	900

Áp dụng SPLIT với bốn điểm cắt (Bảng 2.5): điểm cắt sau HanMucNo cho z lớn nhất.

Bảng 2.5: *Giá trị z tại các điểm cắt*

Nhóm trên TA	**CTQ**	**CBQ**	**COQ**	**z**
{CongNo}	30	1350	80	34.100
{CongNo, HanMucNo}	110	1350	0	**148.500**
{CongNo, HanMucNo, DiaChi}	110	0	1350	−1.822.500
{CongNo, HanMucNo, DiaChi, HoTen}	560	0	900	−810.000

Kết quả là hai mảnh dọc, mỗi mảnh đều chứa khóa MaKH:

$$\text{KH_TC} = \pi_{\text{MaKH, HanMucNo, CongNo}}(\text{KhachHang}),\quad \text{KH_GD} = \pi_{\text{MaKH, HoTen, SDT, DiaChi}}(\text{KhachHang})$$

Mảnh KH_GD phục vụ các ứng dụng tại chi nhánh; mảnh KH_TC phục vụ phòng kế toán tại trụ sở. Không có ứng dụng nào phải dùng cả hai mảnh (COQ = 0).

2.6 Cấp phát và nhân bản

Sau khi phân mảnh, mỗi mảnh cần được đặt lên một hoặc nhiều site. Bài toán cấp phát tối ưu là bài toán NP-khó, nên trong thực tế thường dùng các nguyên tắc heuristic:

- Đặt mảnh tại site truy cập nó nhiều nhất.
- Mảnh chủ yếu được đọc và ít được cập nhật thì nên **nhân bản** ra nhiều site để tăng tính cục bộ và độ sẵn sàng.
- Mảnh được cập nhật thường xuyên thì nên giữ **một bản duy nhất**, tránh chi phí đồng bộ giữa các bản sao.
- Các mảnh thường được kết nối với nhau (như các mảnh dẫn xuất) nên đặt cùng site.

2.7 Kết chương

Chương 2 đã trình bày các quy tắc đúng đắn, phân mảnh ngang nguyên thủy với COM_MIN, phân mảnh ngang dẫn xuất và phân mảnh dọc với BEA và SPLIT, kèm ví dụ trên dữ liệu quản lý khách hàng. Các ví dụ cho thấy KhachHang vừa cần chia theo chi nhánh, vừa cần tách nhóm thuộc tính giao dịch và tài chính. Để thỏa đồng thời cả hai nhu cầu cần đến phân mảnh hỗn hợp, nội dung của Chương 3.

CHƯƠNG 3. PHÂN MẢNH HỖN HỢP

3.1 Định nghĩa và động cơ sử dụng

3.1.1 Động cơ

Phân mảnh ngang phù hợp khi các ứng dụng khác nhau ở **những bộ** mà chúng truy cập. Phân mảnh dọc phù hợp khi các ứng dụng khác nhau ở **những thuộc tính** mà chúng truy cập. Trong nhiều hệ thống thực tế, hai sự khác biệt này xuất hiện đồng thời. Bảng 3.1 mô tả nhu cầu truy cập quan hệ KhachHang của các nhóm người dùng trong một công ty có ba chi nhánh.

Bảng 3.1: *Nhu cầu truy cập KhachHang theo dòng và theo cột*

Nhóm người dùng	Những bộ cần truy cập	Những thuộc tính cần truy cập
Nhân viên chi nhánh Hà Nội	Khách của chi nhánh Hà Nội	Nhóm giao dịch: HoTen, SDT, Email, DiaChi
Nhân viên chi nhánh Đà Nẵng	Khách của chi nhánh Đà Nẵng	Nhóm giao dịch
Nhân viên chi nhánh TP.HCM	Khách của chi nhánh TP.HCM	Nhóm giao dịch
Phòng kế toán (trụ sở)	Khách của cả ba chi nhánh, báo cáo theo từng miền	Nhóm tài chính: HanMucNo, CongNo, DiemTichLuy

Nếu chỉ phân mảnh ngang theo chi nhánh, mỗi chi nhánh vẫn phải lưu các cột tài chính mà mình không dùng, và phòng kế toán phải truy cập cả ba chi nhánh để lấy công nợ. Nếu chỉ phân mảnh dọc, mỗi chi nhánh phải lưu khách hàng của các chi nhánh khác. Không kiểu phân mảnh đơn lẻ nào đáp ứng được cả hai nhu cầu.

3.1.2 Định nghĩa

Phân mảnh hỗn hợp (hybrid fragmentation, còn gọi là phân mảnh lai) là việc áp dụng **nối tiếp** phân mảnh ngang và phân mảnh dọc lên một quan hệ: kết quả của một bước phân mảnh lại được phân mảnh tiếp theo kiểu còn lại [1], [2]. Một mảnh hỗn hợp vì vậy được biểu diễn bằng biểu thức kết hợp phép chọn và phép chiếu, có hai dạng:

$$R_{ij} = \pi_{K \cup A_j}(\sigma_{F_i}(R)) \qquad \text{hoặc} \qquad R_{jk} = \sigma_{F_k}(\pi_{K \cup A_j}(R))$$

trong đó K là khóa chính của R, A_j là một nhóm thuộc tính, F_i và F_k là các vị từ chọn. Về hình ảnh, nếu coi quan hệ là một bảng thì mỗi mảnh hỗn hợp là một ô trên lưới được cắt cả theo dòng lẫn theo cột (Hình 2.1, phần bên phải).

Về lý thuyết, quá trình phân mảnh có thể lồng nhiều mức. Trong thực tế, mức lồng thường không quá hai, vì mỗi mức tăng thêm làm chi phí tái thiết tăng theo và lợi ích thu được giảm dần.

3.2 Chiến lược ngang trước, dọc sau (HV)

Trong chiến lược HV, quan hệ được phân mảnh ngang trước, sau đó **từng** mảnh ngang được phân mảnh dọc:

1. Phân mảnh ngang nguyên thủy R thành R_1, …, R_n với $R_i = \sigma_{F_i}(R)$, thiết kế bằng COM_MIN như ở mục 2.3.
2. Với mỗi R_i, xác định các nhóm thuộc tính bằng ma trận ái lực, BEA và SPLIT (mục 2.5), tính trên các ứng dụng truy cập R_i, rồi chia R_i thành các mảnh $R_{ij} = \pi_{K \cup A_{ij}}(R_i)$.

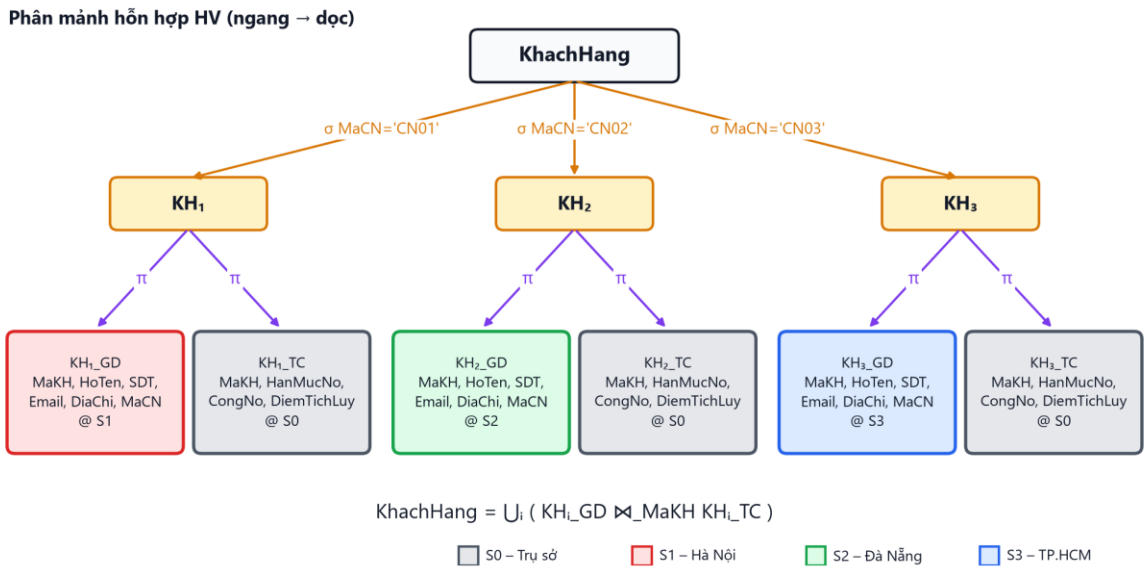
Đặc điểm của HV:

– Mỗi mảnh ngang có thể được chia dọc theo **cách khác nhau**, vì các ứng dụng truy cập mỗi mảnh ngang có thể khác nhau. Ví dụ, một chi nhánh triển khai chương trình khách hàng thân thiết có thể tách thêm nhóm thuộc tính điểm thưởng, các chi nhánh khác thì không.

– Phù hợp khi tiêu chí phân biệt **chủ yếu là theo bộ** (theo địa lý, theo đơn vị quản lý), còn sự khác biệt theo thuộc tính là thứ yếu và có thể phụ thuộc vào từng nhóm bộ.

– Một nhóm thuộc tính, dù được đặt tập trung, vẫn bị chia theo vị từ ngang. Điều này có lợi khi các ứng dụng tập trung cũng làm việc theo từng nhóm bộ, chẳng hạn báo cáo công nợ theo từng miền.

Hình 3.1 là cây phân mảnh HV của quan hệ KhachHang trong bài toán ở Chương 4: ba mảnh ngang theo chi nhánh, mỗi mảnh tách thành phần giao dịch (GD) và phần tài chính (TC), tổng cộng sáu mảnh lá.



Hình 3.1: *Cây phân mảnh hỗn hợp HV của quan hệ KhachHang*

3.3 Chiến lược dọc trước, ngang sau (VH)

Trong chiến lược VH, quan hệ được phân mảnh dọc trước, sau đó **một số** mảnh dọc được phân mảnh ngang tiếp:

1. Phân mảnh dọc R thành các mảnh $R_j = \pi_{K \cup A_j}(R)$ bằng BEA và SPLIT, tính trên toàn bộ ứng dụng.
2. Với những mảnh dọc mà các ứng dụng sử dụng chúng chỉ truy cập một phần các bộ, phân mảnh ngang tiếp: $R_{jk} = \sigma_{F_{jk}}(R_j)$.

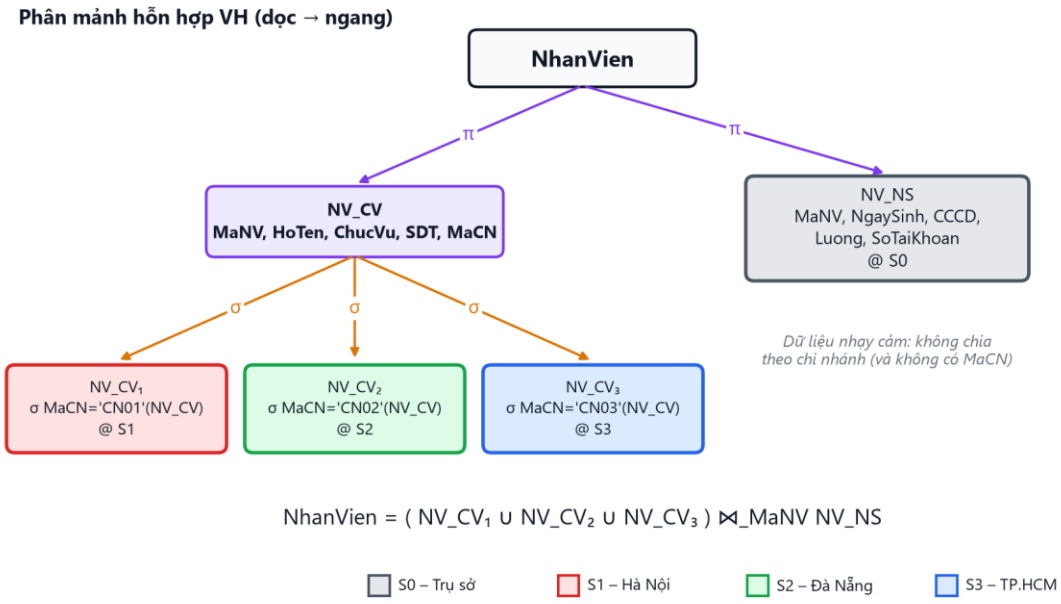
Đặc điểm của VH:

– Chỉ những mảnh dọc **cần thiết** mới bị chia ngang; các mảnh dọc khác được giữ nguyên. Nhờ vậy số mảnh ít hơn và việc quản lý đơn giản hơn.

– Phù hợp khi tiêu chí phân biệt **chủ yếu là theo thuộc tính**, chẳng hạn tách dữ liệu nhạy cảm để bảo mật. Phần nhạy cảm thường được xử lý tập trung nên không cần chia ngang.

– **Ràng buộc:** vị từ F_{jk} chỉ được dùng các thuộc tính có mặt trong mảnh R_j. Nếu mảnh dọc cần chia không chứa thuộc tính của vị từ, ta phải chia nó bằng phân mảnh ngang dẫn xuất theo một mảnh dọc khác (nửa kết nối trên khóa K), hoặc đưa thuộc tính đó vào mảnh dọc từ bước 1.

Hình 3.2 là cây phân mảnh VH của quan hệ NhanVien. Thông tin nhân sự NV_NS (ngày sinh, CCCD, lương, số tài khoản) được tách ra và giữ tập trung tại trụ sở. Chỉ phần thông tin công việc NV_CV được chia tiếp theo chi nhánh. Mảnh NV_NS không chứa MaCN nên không thể chia ngang trực tiếp theo chi nhánh; đây cũng là một lý do để chọn VH cho quan hệ này.



Hình 3.2: *Cây phân mảnh hỗn hợp VH của quan hệ NhanVien*

3.3.1 Quan hệ giữa HV và VH

Phép chọn và phép chiếu giao hoán với nhau khi các thuộc tính của vị từ nằm trong tập thuộc tính được chiếu:

$$\sigma_F(\pi_A(R)) = \pi_A(\sigma_F(R)) \quad \text{nếu} \quad attr(F) \subseteq A$$

Vì vậy, nếu **mọi** mảnh dọc đều chứa thuộc tính của vị từ và đều được chia ngang theo **cùng** các vị từ, HV và VH cho cùng một tập mảnh lá. Hai chiến lược chỉ thực sự khác nhau khi cây phân mảnh **không cân đối**: chỉ một số nút được chia tiếp (VH), hoặc mỗi nút được chia theo cách khác nhau (HV). Do đó, khi chọn chiến lược, câu hỏi cần trả lời là: tiêu chí nào phân biệt các ứng dụng rõ hơn và nên được áp dụng trước, và phần dữ liệu nào không cần chia tiếp.

3.4 Cây phân mảnh và biểu diễn bằng đại số quan hệ

Cây phân mảnh là cách biểu diễn trực quan một lược đồ phân mảnh hỗn hợp:

- **Gốc** là quan hệ toàn cục R.
- Mỗi **nút trong** là một mảnh trung gian. Các cạnh từ một nút trong xuống các con của nó đều cùng một kiểu phân mảnh: hoặc đều là phép chọn σ (chia ngang), hoặc đều là phép chiếu π (chia dọc).
- Mỗi **lá** là một mảnh cuối cùng, được lưu trữ vật lý và cấp phát cho site.

Biểu thức đại số của một mảnh lá có được bằng cách hợp thành các phép toán dọc đường đi từ gốc đến lá. Ví dụ, ở Hình 3.1, lá KH_1_GD có biểu thức:

$$KH_1_GD = \pi_{MaKH,\ HoTen,\ SDT,\ Email,\ DiaChi,\ MaCN}(\sigma_{MaCN='CN01'}(KhachHang))$$

Ở Hình 3.2, lá NV_CV_2 có biểu thức:

$$NV_CV_2 = \sigma_{MaCN='CN02'}(\pi_{MaNV,\ HoTen,\ ChucVu,\ SDT,\ MaCN}(\text{NhanVien}))$$

3.5 Tái thiết quan hệ toàn cục

Quan hệ toàn cục được tái thiết bằng cách duyệt cây phân mảnh **từ lá lên gốc**. Tại mỗi nút trong:

- nếu các con được tạo bằng phép chọn σ: nút = **hợp (∪)** các con;
- nếu các con được tạo bằng phép chiếu π: nút = **kết nối (⋈) trên khóa K** các con.

Áp dụng cho hai chiến lược ta có công thức tổng quát:

$$\text{HV:}\quad R = \bigcup_i (R_{i1} \bowtie_K R_{i2} \bowtie_K \dots \bowtie_K R_{im})$$

$$\text{VH:}\quad R = (R_{11} \cup R_{12} \cup \dots \cup R_{1n}) \bowtie_K R_2 \bowtie_K \dots \bowtie_K R_m$$

Với hai ví dụ ở trên:

$$\text{KhachHang} = \bigcup_{i=1..3} (KH_i_GD \bowtie_{MaKH} KH_i_TC)$$

$$\text{NhanVien} = (NV_CV_1 \cup NV_CV_2 \cup NV_CV_3) \bowtie_{MaNV} NV_NS$$

Trong cài đặt bằng SQL, phép hợp các mảnh ngang rời nhau có thể dùng UNION ALL thay cho UNION. Vì các mảnh đã rời nhau, không có bộ trùng cần loại bỏ, nên UNION ALL cho cùng kết quả mà không tốn chi phí sắp xếp hay băm để khử trùng lặp.

3.6 Kiểm tra tính đúng đắn

Mệnh đề. Nếu phân mảnh tại mọi nút trong của cây phân mảnh đều thỏa ba quy tắc đầy đủ, tái thiết và rời nhau, thì phân mảnh hỗn hợp tương ứng với toàn bộ cây cũng thỏa ba quy tắc đó.

Chứng minh (phác thảo) bằng quy nạp theo chiều cao của cây:

- **Đầy đủ:** mỗi mục dữ liệu của nút cha có mặt trong ít nhất một nút con (giả thiết tại nút cha). Theo giả thiết quy nạp, mỗi mục dữ liệu của nút con có mặt trong ít nhất một lá của cây con. Vậy mọi mục dữ liệu của R đều có mặt trong ít nhất một lá.
- **Tái thiết:** mỗi nút con tái thiết được từ các lá của nó (giả thiết quy nạp), và nút cha tái thiết được từ các con bằng ∪ hoặc ⋈ (giả thiết tại nút cha). Hợp thành các phép toán đó cho biểu thức tái thiết R từ các lá, như ở mục 3.5.
- **Rời nhau:** hai lá bất kỳ có một nút tổ tiên chung gần nhất, và chúng nằm trong hai cây con của hai nút con khác nhau. Nếu nút tổ tiên đó chia ngang, hai lá chứa

các tập bộ rời nhau. Nếu nút đó chia dọc, hai lá chỉ có chung thuộc tính khóa K. Trường hợp nào thì quy tắc rời nhau cũng thỏa.

Áp dụng cho quan hệ KhachHang (HV): bước ngang thỏa ba quy tắc như đã chỉ ra ở mục 2.3.4; bước dọc trên mỗi KH_i thỏa vì GD ∪ TC chứa đủ mọi thuộc tính, GD ∩ TC = {MaKH} là khóa, và kết nối trên khóa không làm mất hay sinh thêm bộ. Do đó cả sáu mảnh lá là một phân mảnh đúng đắn. Lập luận tương tự áp dụng cho NhanVien (VH). Ở Chương 4, ba quy tắc này còn được kiểm tra trên dữ liệu thực bằng truy vấn SQL.

3.7 Cấp phát mảnh hỗn hợp và ảnh hưởng đến xử lý

3.7.1 Cấp phát

Chỉ các **lá** của cây phân mảnh được cấp phát. Ví dụ, các mảnh GD của KhachHang đặt tại chi nhánh tương ứng, các mảnh TC đặt tại trụ sở. Nguyên tắc cấp phát vẫn như ở mục 2.6: đặt mảnh tại nơi dùng nhiều nhất, đặt cùng site các mảnh thường được kết nối với nhau, nhân bản các mảnh chủ yếu được đọc.

3.7.2 Ảnh hưởng đến xử lý truy vấn

Khi nhận một truy vấn trên quan hệ toàn cục, DDBMS thay quan hệ đó bằng biểu thức tái thiết (gọi là **chương trình định vị**, localization program), rồi rút gọn [1]:

- **Rút gọn theo phân mảnh ngang:** loại các nhánh có vị từ mâu thuẫn với điều kiện của truy vấn.
- **Rút gọn theo phân mảnh dọc:** loại các mảnh dọc không chứa thuộc tính nào mà truy vấn cần, ngoài khóa.
- **Rút gọn theo phân mảnh dẫn xuất:** phân phối phép kết nối qua phép hợp và loại các cặp mảnh không thể kết nối được với nhau.

Với phân mảnh hỗn hợp, các quy tắc trên được áp dụng kết hợp. Ví dụ với quan hệ KhachHang:

- Truy vấn SELECT HoTen, SDT FROM KhachHang WHERE MaCN = 'CN01' chỉ cần mảnh KH_1_GD, chạy hoàn toàn tại chi nhánh Hà Nội.
- Truy vấn SELECT SUM(CongNo) FROM KhachHang WHERE MaCN = 'CN02' chỉ cần mảnh KH_2_TC, chạy hoàn toàn tại trụ sở.
- Truy vấn SELECT HoTen, CongNo FROM KhachHang WHERE MaCN = 'CN02' cần KH_2_GD ⋈ KH_2_TC: hai mảnh nằm ở hai site khác nhau, phải truyền dữ liệu qua mạng.

3.7.3 Ảnh hưởng đến cập nhật và giao dịch

- Một thao tác cập nhật chạm đến thuộc tính thuộc nhiều mảnh dọc (ví dụ lập đơn hàng ở chi nhánh và cộng công nợ ở trụ sở) trở thành **giao dịch phân tán**, cần giao thức cam kết hai pha (2PC) để đảm bảo tính nguyên tử.
- Thêm một bộ mới phải ghi vào **tất cả** các mảnh dọc tương ứng, để không vi phạm tính tái thiết.
- Cập nhật **thuộc tính phân mảnh** (ví dụ khách hàng chuyển chi nhánh, thay đổi MaCN) làm bộ phải di chuyển từ mảnh này sang mảnh khác, ở tất cả các nhánh dọc, kèm theo các mảnh dẫn xuất.

3.8 Ưu điểm, hạn chế và so sánh

Ưu điểm của phân mảnh hỗn hợp:

- Mỗi mảnh khớp sát với nhu cầu của một nhóm ứng dụng, nên tính cục bộ cao nhất trong ba kiểu phân mảnh.
- Kết hợp được mục tiêu hiệu năng (chia theo dòng) và bảo mật (tách cột nhạy cảm).
- Giảm khối lượng dữ liệu lưu trữ và truyền tại mỗi site.

Hạn chế:

- Thiết kế phức tạp: phải phân tích cả vị từ lẫn ái lực thuộc tính, và chọn thứ tự áp dụng.
- Số mảnh lớn (tích số mảnh ngang và số nhóm thuộc tính), nên khó quản lý hơn.
- Truy vấn cần đầy đủ thông tin phải dùng cả $\cup$ và $\bowtie$, chi phí tái thiết cao.
- Cập nhật dễ trở thành giao dịch phân tán.
- Hầu hết các DBMS thương mại không hỗ trợ trực tiếp, phải tự cài đặt.

Bảng 3.2 tóm tắt so sánh ba kiểu phân mảnh.

Bảng 3.2: *So sánh ba kiểu phân mảnh*

Tiêu chí	Phân mảnh ngang	Phân mảnh dọc	Phân mảnh hỗn hợp
Đơn vị chia	Tập bộ	Tập thuộc tính + khóa	Tập bộ × tập thuộc tính
Phép toán	σ (hoặc $\ltimes$ nếu dẫn xuất)	π	Kết hợp σ và π
Tái thiết	$\cup$	$\bowtie$ trên khóa	$\cup$ và $\bowtie$ theo cây phân mảnh
Rời nhau	Các tập bộ rời nhau	Chỉ trùng ở khóa	Theo nút tổ tiên chung

Tiêu chí	Phân mảnh ngang	Phân mảnh dọc	Phân mảnh hỗn hợp
Công cụ thiết kế	Vị từ, minterm, COM_MIN	Ma trận ái lực, BEA, SPLIT	Cả hai, theo thứ tự HV hoặc VH
Phù hợp khi	Ứng dụng khác nhau theo bộ (địa lý, đơn vị)	Ứng dụng khác nhau theo thuộc tính, cần bảo mật cột	Ứng dụng khác nhau theo cả bộ lẫn thuộc tính
Độ phức tạp	Thấp	Trung bình	Cao

3.9 Hỗ trợ trong các hệ quản trị thực tế

Các DBMS phổ biến chủ yếu hỗ trợ **phân mảnh ngang**. Phân mảnh dọc và phân mảnh hỗn hợp thường phải tự xây dựng bằng các bảng riêng và view tái thiết.

- **Microsoft SQL Server:** *table partitioning* chia một bảng theo dòng dựa trên một cột phân hoạch, nhưng các phân hoạch nằm trong cùng một CSDL. Để phân tán sang nhiều máy, SQL Server cung cấp **Linked Server** (truy cập bảng ở máy chủ khác bằng tên bốn phần) và **Distributed Partitioned View**: các bảng thành viên ở nhiều máy, mỗi bảng có ràng buộc CHECK trên cột phân hoạch, được hợp lại bằng một view UNION ALL. Nhờ ràng buộc CHECK, bộ tối ưu truy vấn bỏ qua được các bảng thành viên không liên quan [3], [6]. **Synonym** cho phép đặt tên cục bộ cho đối tượng ở xa, giúp đạt tính trong suốt vị trí [7].
- **Oracle:** *Oracle Partitioning* hỗ trợ phân hoạch theo khoảng (range), danh sách (list), băm (hash) và phân hoạch kết hợp (composite). Cần lưu ý phân hoạch kết hợp vẫn là chia ngang hai mức, không phải phân mảnh hỗn hợp. Việc phân tán dùng *database link* kết hợp với view.
- **PostgreSQL:** hỗ trợ *declarative partitioning* (range, list, hash), trong đó một phân hoạch có thể là *foreign table* trỏ sang máy khác qua *postgres_fdw*. Phần mở rộng **Citus** phân mảnh ngang (sharding) bảng trên nhiều nút, và cho phép khai báo *reference table* được nhân bản ra mọi nút.

Như vậy, để cài đặt phân mảnh hỗn hợp, cách phổ biến là: dùng cơ chế chia ngang (bảng riêng có CHECK, hoặc partitioning) cho bước ngang; dùng các bảng riêng chứa khóa chung cho bước dọc; và dùng view kết hợp UNION ALL và JOIN để tái thiết. Đây là cách nhóm áp dụng ở Chương 4.

3.10 Kết chương

Chương 3 đã trình bày phân mảnh hỗn hợp: động cơ, định nghĩa, hai chiến lược HV và VH cùng quan hệ giữa chúng, cây phân mảnh, công thức tái thiết, chứng minh tính đúng đắn, ảnh hưởng đến cấp phát, truy vấn và giao dịch, cũng như hỗ trợ trong các

DBMS thực tế. Chương 4 áp dụng các nội dung này để thiết kế và cài đặt CSDL phân tán cho hệ thống quản lý đơn hàng.

CHƯƠNG 4. ỨNG DỤNG PHÂN MẢNH HỖN HỢP CHO HỆ THỐNG QUẢN LÝ ĐƠN HÀNG

4.1 Mô tả bài toán

Công ty thương mại X (công ty giả định) phân phối thiết bị văn phòng qua ba chi nhánh ở ba miền và có một trụ sở chính đặt tại TP. Hồ Chí Minh. Mỗi địa điểm là một site của CSDL phân tán (Bảng 4.1).

Bảng 4.1: *Các site trong hệ thống*

Site	Địa điểm	Mã chi nhánh	Người dùng chính
S0	Trụ sở chính (TP.HCM)	—	Phòng Kế toán, phòng Nhân sự, ban Giám đốc
S1	Chi nhánh Hà Nội (miền Bắc)	CN01	Nhân viên bán hàng, quản lý chi nhánh
S2	Chi nhánh Đà Nẵng (miền Trung)	CN02	Nhân viên bán hàng, quản lý chi nhánh
S3	Chi nhánh TP.HCM (miền Nam)	CN03	Nhân viên bán hàng, quản lý chi nhánh

Các nghiệp vụ chính:

- Mỗi khách hàng được quản lý bởi **đúng một** chi nhánh, là nơi khách đăng ký.
- Nhân viên chi nhánh tra cứu khách hàng, lập đơn hàng và cập nhật trạng thái đơn cho khách của chi nhánh mình. Đây là nghiệp vụ diễn ra nhiều nhất.
- Phòng Kế toán quản lý hạn mức công nợ, công nợ hiện tại và điểm tích lũy của khách hàng ở cả ba miền, lập báo cáo công nợ theo từng miền.
- Phòng Nhân sự quản lý ngày sinh, số CCCD, lương và số tài khoản của nhân viên. Đây là dữ liệu nhạy cảm, chi nhánh không được truy cập.
- Ban Giám đốc xem báo cáo doanh thu toàn công ty và theo từng chi nhánh.
- Danh mục sản phẩm và giá bán do trụ sở quản lý; mọi chi nhánh đều cần đọc khi lập đơn.

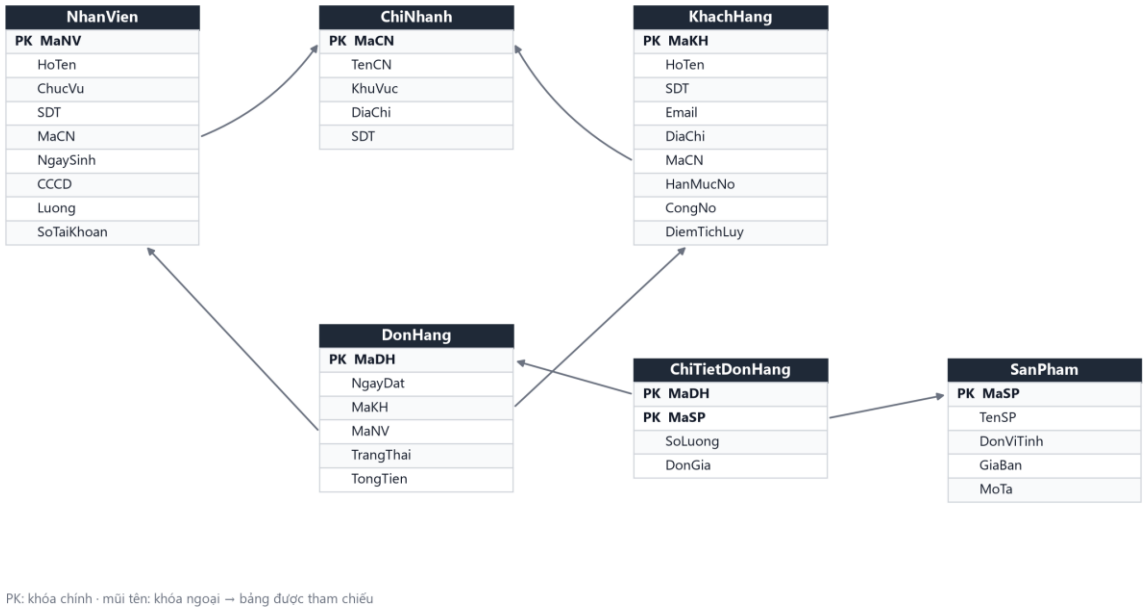
Giả định: đơn hàng của khách nào thì do chi nhánh quản lý khách đó xử lý, và nhân viên lập đơn thuộc cùng chi nhánh; nhân viên làm việc tại trụ sở nằm ngoài phạm vi bài toán.

4.2 Lược đồ toàn cục

Lược đồ toàn cục gồm sáu quan hệ (khóa chính được gạch chân trong sơ đồ, đánh dấu PK trong Hình 4.1):

- ChiNhanh(**MaCN**, TenCN, KhuVuc, DiaChi, SDT)
- KhachHang(**MaKH**, HoTen, SDT, Email, DiaChi, MaCN, HanMucNo, CongNo, DiemTichLuy)
- NhanVien(**MaNV**, HoTen, ChucVu, SDT, MaCN, NgaySinh, CCCD, Luong, SoTaiKhoan)
- SanPham(**MaSP**, TenSP, DonViTinh, GiaBan, MoTa)
- DonHang(**MaDH**, NgayDat, MaKH, MaNV, TrangThai, TongTien)
- ChiTietDonHang(**MaDH, MaSP**, SoLuong, DonGia)

Ràng buộc miền giá trị MaCN ∈ {'CN01', 'CN02', 'CN03'} được khai báo bằng CHECK. Ràng buộc này là cơ sở để chứng minh tính đầy đủ của phân mảnh ngang.



Hình 4.1: *Lược đồ quan hệ toàn cục của hệ thống quản lý đơn hàng*

4.3 Phân tích ứng dụng

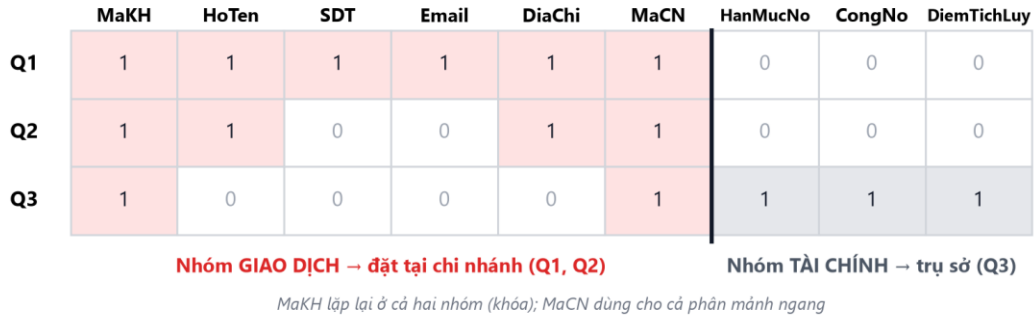
Theo phương pháp thiết kế từ trên xuống, nhóm liệt kê các ứng dụng chính, nơi phát sinh, dữ liệu sử dụng và tần suất (Bảng 4.2).

Bảng 4.2: *Các ứng dụng chính của hệ thống*

Mã	Ứng dụng	Phát sinh tại	Dữ liệu sử dụng	Tần suất
Q1	Tra cứu khách hàng của chi nhánh để lập đơn	S1, S2, S3	KhachHang(MaKH, HoTen, SDT, Email, DiaChi, MaCN)	Rất cao
Q2	Lập đơn, xem và cập nhật đơn hàng của chi nhánh	S1, S2, S3	DonHang, ChiTietDonHang, SanPham	Rất cao

Mã	Ứng dụng	Phát sinh tại	Dữ liệu sử dụng	Tần suất
Q3	Kiểm tra hạn mức, công nợ; báo cáo công nợ theo miền	S0	KhachHang(MaKH, MaCN, HanMucNo, CongNo, DiemTichLuy)	Trung bình
Q4	Xem danh sách nhân viên chi nhánh	S1, S2, S3	NhanVien(MaNV, HoTen, ChucVu, SDT, MaCN)	Trung bình
Q5	Tính lương, quản lý hồ sơ nhân sự	S0	NhanVien(MaNV, NgaySinh, CCCD, Luong, SoTaiKhoan)	Thấp
Q6	Báo cáo doanh thu toàn công ty, theo chi nhánh	S0	DonHang, ChiTietDonHang	Thấp
Q7	Cập nhật danh mục và giá sản phẩm	S0	SanPham	Thấp

Ma trận sử dụng thuộc tính của KhachHang (Hình 4.2) cho thấy các thuộc tính chia thành hai nhóm tách biệt: **nhóm giao dịch** do Q1, Q2 sử dụng tại chi nhánh và **nhóm tài chính** do Q3 sử dụng tại trụ sở. Hai nhóm chỉ chung khóa MaKH và thuộc tính MaCN. Kết quả này phù hợp với ví dụ BEA và SPLIT ở mục 2.5.5. Đồng thời, mỗi chi nhánh chỉ truy cập các bộ của khách thuộc chi nhánh mình. Như vậy KhachHang cần được phân mảnh theo cả bộ lẫn thuộc tính. Lập luận tương tự áp dụng cho NhanVien: Q4 dùng thông tin công việc tại chi nhánh, Q5 dùng thông tin nhân sự tại trụ sở.



Hình 4.2: *Ma trận sử dụng thuộc tính của quan hệ KhachHang*

4.4 Thiết kế phân mảnh

Bảng 4.3 tóm tắt kiểu phân mảnh được chọn cho từng quan hệ. Chi tiết được trình bày trong các mục tiếp theo.

Bảng 4.3: *Kiểu phân mảnh của từng quan hệ*

Quan hệ	Kiểu phân mảnh	Số mảnh lá	Lý do
KhachHang	Hỗn hợp HV	6	Chi nhánh cần dòng của mình + cột giao dịch; kế toán cần cột tài chính, báo cáo theo miền

Quan hệ	Kiểu phân mảnh	Số mảnh lá	Lý do
NhanVien	Hỗn hợp VH	4	Tách dữ liệu nhân sự nhạy cảm, xử lý tập trung; chỉ chia ngang phần công việc
DonHang	Ngang dẫn xuất theo KhachHang	3	Đặt đơn cùng site với khách, kết nối cục bộ
ChiTietDonHang	Ngang dẫn xuất theo DonHang	3	Đặt chi tiết cùng site với đơn
ChiNhanh, SanPham	Không phân mảnh, nhân bản toàn phần	—	Nhỏ, chủ yếu được đọc, ít cập nhật

4.4.1 KhachHang – phân mảnh hỗn hợp HV

Bước 1 – phân mảnh ngang nguyên thủy theo các vị từ p_i: MaCN = 'CN0i' (đã thiết kế ở mục 2.3.4):

$$\mathrm{KH_i} = \sigma_{\mathrm{MaCN='CN0i'}}(\mathrm{KhachHang}),\ \ \mathrm{i} = 1, 2, 3$$

Bước 2 – phân mảnh dọc từng mảnh $\mathrm{KH_i}$ thành phần giao dịch và phần tài chính. Thuộc tính MaCN được giữ ở phần giao dịch vì đó là thuộc tính phân mảnh, cần cho ràng buộc CHECK tại chi nhánh:

$$\mathrm{KH_i_GD} = \pi_{\mathrm{MaKH,\ HoTen,\ SDT,\ Email,\ DiaChi,\ MaCN}}(\mathrm{KH_i}) \quad \rightarrow \text{đặt tại } \mathrm{S_i}$$

$$\mathrm{KH_i_TC} = \pi_{\mathrm{MaKH,\ HanMucNo,\ CongNo,\ DiemTichLuy}}(\mathrm{KH_i}) \quad \rightarrow \text{đặt tại } \mathrm{S0}$$

Tái thiết:

$$\mathrm{KhachHang} = \bigcup_{\mathrm{i=1..3}} (\ \mathrm{KH_i_GD} \bowtie_{\mathrm{MaKH}} \mathrm{KH_i_TC}\ )$$

Cây phân mảnh gồm sáu lá, như Hình 3.1. Nhóm chọn HV thay vì VH vì phòng Kế toán lập báo cáo công nợ **theo từng miền**. Phần tài chính được chia sẵn theo miền nên báo cáo của mỗi miền chỉ cần đọc đúng một mảnh. Mảnh TC không cần chứa MaCN: miền của khách đã được xác định bởi chính mảnh chứa nó, nhờ phép chọn ở bước 1.

4.4.2 NhanVien – phân mảnh hỗn hợp VH

Bước 1 – phân mảnh dọc thành thông tin công việc và thông tin nhân sự:

$$\mathrm{NV_CV} = \pi_{\mathrm{MaNV,\ HoTen,\ ChucVu,\ SDT,\ MaCN}}(\mathrm{NhanVien})$$

$$\mathrm{NV_NS} = \pi_{\mathrm{MaNV,\ NgaySinh,\ CCCD,\ Luong,\ SoTaiKhoan}}(\mathrm{NhanVien}) \quad \rightarrow \text{đặt tại } \mathrm{S0}$$

Bước 2 – phân mảnh ngang chỉ trên mảnh công việc:

$$\mathrm{NV_CV_i} = \sigma_{\mathrm{MaCN='CN0i'}}(\mathrm{NV_CV}) \quad \rightarrow \text{đặt tại } \mathrm{S_i},\ \ \mathrm{i} = 1, 2, 3$$

Tái thiết:

$$\text{NhanVien} = (\ \text{NV_CV}_1 \cup \text{NV_CV}_2 \cup \text{NV_CV}_3\ ) \bowtie_{\text{MaNV}} \text{NV_NS}$$

Cây phân mảnh gồm bốn lá, như Hình 3.2. Nhóm chọn VH vì tiêu chí phân biệt chính ở đây là **thuộc tính**, nhằm tách dữ liệu nhạy cảm. Phần nhân sự được xử lý tập trung tại trụ sở nên không cần chia ngang; hơn nữa NV_NS không chứa MaCN nên cũng không chia ngang trực tiếp theo chi nhánh được. Việc dùng HV cho KhachHang và VH cho NhanVien cũng cho phép so sánh hai chiến lược trên cùng một hệ thống.

4.4.3 DonHang, ChiTietDonHang – phân mảnh ngang dẫn xuất

$$\text{DH}_i = \text{DonHang} \ltimes_{\text{MaKH}} \text{KH}_i, \quad \text{CTDH}_i = \text{ChiTietDonHang} \ltimes_{\text{MaDH}} \text{DH}_i, \quad i = 1, 2, 3$$

Trong đó KH_i là mảnh ngang ở bước 1 của mục 4.4.1, có chứa MaKH. Nhờ phân mảnh dẫn xuất, mọi phép kết nối giữa khách hàng, đơn hàng và chi tiết đơn hàng đều thực hiện được tại chỗ ở chi nhánh.

4.4.4 ChiNhanh, SanPham – nhân bản toàn phần

Hai quan hệ này nhỏ (3 và 10 bộ trong dữ liệu mẫu), được đọc ở mọi site và chỉ trụ sở cập nhật. Chúng được nhân bản toàn phần ở cả bốn site. Mọi cập nhật thực hiện tại S0 rồi đồng bộ xuống các chi nhánh.

4.5 Kiểm tra tính đúng đắn

Bảng 4.4 tóm tắt việc kiểm tra ba quy tắc cho hai quan hệ được phân mảnh hỗn hợp.

Bảng 4.4: *Kiểm tra tính đúng đắn của phân mảnh hỗn hợp*

Quy tắc	KhachHang (HV)	NhanVien (VH)
Đầy đủ	Bước ngang: {MaCN='CN01', 'CN02', 'CN03'} phủ toàn miền giá trị MaCN (có CHECK). Bước dọc: GD ∪ TC = toàn bộ thuộc tính	Bước dọc: CV ∪ NS = toàn bộ thuộc tính. Bước ngang: các vị từ phủ miền giá trị MaCN
Tái thiết	⋈ trên khóa MaKH không mất, không thừa bộ; ∪ các kết quả cho lại KhachHang	∪ các NV_CV_i cho lại NV_CV; ⋈ với NV_NS trên khóa MaNV
Rời nhau	Các vị từ loại trừ nhau nên các KH_i rời nhau; GD ∩ TC = {MaKH}	Các NV_CV_i rời nhau; CV ∩ NS = {MaNV}

Với DonHang và ChiTietDonHang, tính đầy đủ được đảm bảo bởi ràng buộc khóa ngoại. Tính rời nhau được đảm bảo vì mỗi đơn hàng có đúng một khách hàng, và mỗi khách hàng thuộc đúng một chi nhánh. Ngoài lập luận lý thuyết, cả ba quy tắc được kiểm tra trên dữ liệu bằng truy vấn SQL ở mục 4.7.3.

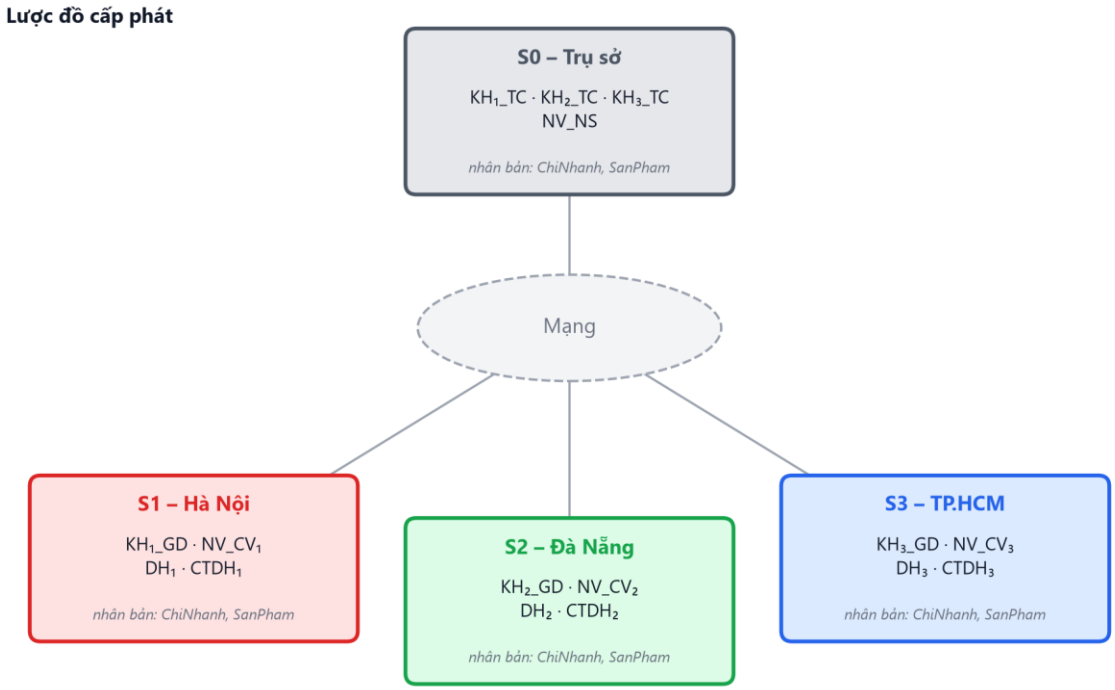
4.6 Lược đồ cấp phát

Bảng 4.5 và Hình 4.3 trình bày lược đồ cấp phát các mảnh cho bốn site.

Bảng 4.5: *Lược đồ cấp phát*

Site	Mảnh được cấp phát
S0 – Trụ sở	KH_1_TC, KH_2_TC, KH_3_TC, NV_NS, ChiNhanh*, SanPham*
S1 – Hà Nội	KH_1_GD, NV_CV_1, DH_1, $CTDH_1$, ChiNhanh*, SanPham*
S2 – Đà Nẵng	KH_2_GD, NV_CV_2, DH_2, $CTDH_2$, ChiNhanh*, SanPham*
S3 – TP.HCM	KH_3_GD, NV_CV_3, DH_3, $CTDH_3$, ChiNhanh*, SanPham*

(* bản nhân bản)



Hình 4.3: *Lược đồ cấp phát các mảnh cho bốn site*

Với lược đồ cấp phát này, mức độ cục bộ của các ứng dụng như sau:

- **Q1, Q2, Q4** (tần suất cao) chạy **hoàn toàn cục bộ** tại chi nhánh: mọi mảnh cần thiết (KH_i_GD, DH_i, $CTDH_i$, NV_CV_i, SanPham) đều nằm tại S_i.
- **Q3, Q5, Q7** chạy cục bộ tại trụ sở.
- Chỉ **Q6** (báo cáo toàn công ty, tần suất thấp) cần lấy dữ liệu từ ba chi nhánh.

4.7 Cài đặt thực nghiệm trên SQL Server

4.7.1 Môi trường và cách mô phỏng

- Hệ quản trị: Microsoft SQL Server 2022 (bản Developer), công cụ SQL Server Management Studio.
- Mỗi site được mô phỏng bằng một CSDL riêng trên cùng một instance: QLDH_TRUSO (S0), QLDH_HN (S1), QLDH_DN (S2), QLDH_HCM (S3). Thêm CSDL QLDH_GOC chứa lược đồ toàn cục và dữ liệu ban đầu, dùng làm chuẩn để đối chiếu.
- Dữ liệu mẫu: 3 chi nhánh, 30 khách hàng (10 khách mỗi chi nhánh), 12 nhân viên (4 người mỗi chi nhánh), 10 sản phẩm, 45 đơn hàng (15 đơn mỗi chi nhánh) với 97 dòng chi tiết. Dữ liệu được sinh bằng một chương trình Python với seed cố định, nên chạy lại luôn cho cùng kết quả.
- Bốn script được chạy theo thứ tự (Bảng 4.6).

Bảng 4.6: *Các script cài đặt*

Script	Nội dung
01_global_schema.sql	Tạo QLDH_GOC: sáu bảng, khóa, khóa ngoại, CHECK; nạp dữ liệu mẫu
02_fragments.sql	Tạo bốn site, tạo các bảng mảnh và nạp dữ liệu theo đúng biểu thức phân mảnh
03_reconstruct_check.sql	Tạo synonym, view tái thiết; kiểm tra ba quy tắc đúng đắn
04_queries.sql	Truy vấn cục bộ, truy vấn toàn cục, giao dịch phân tán

Việc dùng nhiều CSDL trên một máy là **mô phỏng**: ranh giới giữa các site là ranh giới giữa các CSDL. Thiết kế phân mảnh và cấp phát không phụ thuộc vào hạ tầng này. Như trình bày ở mục 4.7.3, nhờ dùng synonym, có thể chuyển sang nhiều máy thật bằng Linked Server mà không phải sửa view và truy vấn.

4.7.2 Tạo các mảnh

Mỗi mảnh được tạo thành một bảng có khóa chính, và dữ liệu được nạp bằng câu lệnh tương ứng với biểu thức đại số của mảnh. Vị từ phân mảnh được ghi thành ràng buộc CHECK, nên mảnh không thể nhận dữ liệu sai chi nhánh. Ví dụ tại site Hà Nội:

```
-- KH_1_GD = π[MaKH,HoTen,SDT,Email,DiaChi,MaCN]( σ[MaCN='CN01'](KhachHang) )
CREATE TABLE dbo.KH_1_GD (
    MaKH   CHAR(5)       NOT NULL PRIMARY KEY,
    HoTen  NVARCHAR(50)  NOT NULL,
    SDT    VARCHAR(12)   NOT NULL,
    Email  VARCHAR(60)   NULL,
    DiaChi NVARCHAR(100) NOT NULL,
    MaCN   CHAR(4)       NOT NULL,
    CONSTRAINT CK_KH_1_GD_MaCN CHECK (MaCN = 'CN01')
```

```
);
INSERT INTO dbo.KH_1_GD (MaKH, HoTen, SDT, Email, DiaChi, MaCN)
SELECT MaKH, HoTen, SDT, Email, DiaChi, MaCN
FROM QLDH_GOC.dbo.KhachHang
WHERE MaCN = 'CN01';


-- DH_1 = DonHang ⋉[MaKH] KH_1  (phép nửa kết nối viết bằng EXISTS)
INSERT INTO dbo.DH_1 (MaDH, NgayDat, MaKH, MaNV, TrangThai, TongTien)
SELECT d.MaDH, d.NgayDat, d.MaKH, d.MaNV, d.TrangThai, d.TongTien
FROM QLDH_GOC.dbo.DonHang AS d
WHERE EXISTS (SELECT 1 FROM QLDH_GOC.dbo.KhachHang AS k
              WHERE k.MaKH = d.MaKH AND k.MaCN = 'CN01');
```

Sau khi chạy script, số bộ trong các mảnh được thống kê ở Bảng 4.7.

Bảng 4.7: *Số bộ trong các mảnh sau khi phân mảnh*

Site	Mảnh và số bộ
S0 – QLDH_TRUSO	KH_1_TC: 10 · KH_2_TC: 10 · KH_3_TC: 10 · NV_NS: 12
S1 – QLDH_HN	KH_1_GD: 10 · NV_CV_1: 4 · DH_1: 15 · $CTDH_1$: 29
S2 – QLDH_DN	KH_2_GD: 10 · NV_CV_2: 4 · DH_2: 15 · $CTDH_2$: 37
S3 – QLDH_HCM	KH_3_GD: 10 · NV_CV_3: 4 · DH_3: 15 · $CTDH_3$: 31

[ẢNH: demo_01_object_explorer.png – Object Explorer hiển thị 5 CSDL và các bảng mảnh]

[ẢNH: demo_03_manh_GD_TC.png – Nội dung mảnh KH_1_GD (tại S1) và KH_1_TC (tại S0)]

4.7.3 Tái thiết và kiểm tra tính đúng đắn

Tại S0, nhóm tạo **synonym** cho từng mảnh nằm ở chi nhánh, rồi xây dựng **view tái thiết** trên các synonym đó. View v_KhachHang cài đặt đúng công thức tái thiết của chiến lược HV:

```
CREATE SYNONYM dbo.syn_KH_1_GD FOR QLDH_HN.dbo.KH_1_GD;   -- tương tự cho các mảnh khác


-- KhachHang = ∪ i=1..3 ( KH_i_GD ⋉[MaKH] KH_i_TC )
CREATE OR ALTER VIEW dbo.v_KhachHang AS
    SELECT gd.MaKH, gd.HoTen, gd.SDT, gd.Email, gd.DiaChi, gd.MaCN,
           tc.HanMucNo, tc.CongNo, tc.DiemTichLuy
    FROM dbo.syn_KH_1_GD AS gd JOIN dbo.KH_1_TC AS tc ON tc.MaKH = gd.MaKH
    UNION ALL
    SELECT ... FROM dbo.syn_KH_2_GD AS gd JOIN dbo.KH_2_TC AS tc ON tc.MaKH = gd.MaKH
    UNION ALL
    SELECT ... FROM dbo.syn_KH_3_GD AS gd JOIN dbo.KH_3_TC AS tc ON tc.MaKH = gd.MaKH;
```

View v_NhanVien cài đặt công thức của chiến lược VH: hợp ba mảnh NV_CV_i trước, sau đó kết nối với NV_NS. View v_DonHang và v_ChiTietDonHang là phép hợp các mảnh dẫn xuất. Người dùng tại trụ sở truy vấn các view này như truy vấn trên quan hệ toàn cục, không cần biết dữ liệu được chia thành những mảnh nào và nằm ở site nào. Đó là **trong suốt phân mảnh** và **trong suốt vị trí**.

Việc dùng synonym thay vì viết thẳng tên CSDL có một lợi ích quan trọng. Khi chuyển sang chạy trên nhiều máy thật, chỉ cần tạo Linked Server và tạo lại synonym trỏ tới tên bốn phần, chẳng hạn [MAY_HN].QLDH_HN.dbo.KH_1_GD. Các view và truy vấn giữ nguyên.

Để kiểm tra tính đúng đắn, nhóm viết 13 truy vấn so sánh dữ liệu trong các mảnh với CSDL gốc QLDH_GOC:

- **Đầy đủ:** đếm số bộ của quan hệ gốc không thuộc mảnh ngang nào; đếm số thuộc tính của quan hệ gốc không thuộc mảnh dọc nào, bằng cách so sánh danh sách cột trong sys.columns với EXCEPT.
- **Tái thiết:** đếm số bộ lệch giữa quan hệ gốc và view tái thiết theo cả hai chiều bằng EXCEPT, cộng thêm chênh lệch về số lượng bộ.
- **Rời nhau:** đếm số khóa xuất hiện ở nhiều hơn một mảnh ngang bằng GROUP BY … HAVING COUNT(*) > 1; đếm số thuộc tính không khóa có mặt ở cả hai mảnh dọc bằng INTERSECT.

Bảng 4.8 là kết quả kiểm tra. Cả 13 phép kiểm tra đều cho số dòng lệch bằng 0, khẳng định phân mảnh đúng đắn trên dữ liệu thực.

Bảng 4.8: *Kết quả kiểm tra tính đúng đắn*

STT	Quan hệ	Quy tắc	Nội dung kiểm tra	Số lệch	Kết quả
1	KhachHang	Đầy đủ	Bộ không thuộc mảnh ngang nào	0	ĐẠT
2	KhachHang	Đầy đủ	Thuộc tính không thuộc mảnh dọc nào (GD ∪ TC)	0	ĐẠT
3	KhachHang	Tái thiết	Bộ lệch giữa KhachHang gốc và v_KhachHang	0	ĐẠT
4	KhachHang	Rời nhau	MaKH thuộc nhiều hơn một mảnh ngang	0	ĐẠT
5	KhachHang	Rời nhau	Thuộc tính không khóa lặp ở cả GD và TC	0	ĐẠT
6	NhanVien	Đầy đủ	Thuộc tính không thuộc mảnh dọc nào (CV ∪ NS)	0	ĐẠT
7	NhanVien	Đầy đủ	Bộ của NV_CV không thuộc mảnh ngang nào	0	ĐẠT
8	NhanVien	Tái thiết	Bộ lệch giữa NhanVien gốc và v_NhanVien	0	ĐẠT
9	NhanVien	Rời nhau	MaNV thuộc nhiều hơn một mảnh ngang	0	ĐẠT
10	NhanVien	Rời nhau	Thuộc tính không khóa lặp ở cả CV và NS	0	ĐẠT
11	DonHang	Đầy đủ	Đơn hàng không thuộc mảnh nào	0	ĐẠT
12	DonHang	Tái thiết	Bộ lệch giữa DonHang gốc và v_DonHang	0	ĐẠT
13	DonHang	Rời nhau	MaDH thuộc nhiều hơn một mảnh	0	ĐẠT

[ẢNH: demo_05_kiem_tra_dung_dan.png – Kết quả chạy phần kiểm tra trong 03_reconstruct_check.sql]

Ngoài ra, script còn kiểm tra quan hệ ChiTietDonHang (tái thiết từ ba mảnh $CTDH_i$), nằm trong cùng bảng kết quả trên màn hình.

4.7.4 Truy vấn minh họa

Truy vấn cục bộ tại chi nhánh (S1). Truy vấn Q2 liệt kê các đơn đang xử lý của chi nhánh Hà Nội, kèm tên khách, nhân viên và sản phẩm. Nó kết nối năm bảng nhưng tất cả đều nằm tại S1: DH_1, KH_1_GD, NV_CV_1, $CTDH_1$ và bản nhân bản SanPham. Kết quả gồm 6 dòng chi tiết của 4 đơn hàng DH020, DH027, DH028, DH036.

```
SELECT dh.MaDH, dh.NgayDat, kh.HoTen AS KhachHang, sp.TenSP, ct.SoLuong, dh.TrangThai
FROM dbo.DH_1 AS dh
JOIN dbo.KH_1_GD AS kh ON kh.MaKH = dh.MaKH
JOIN dbo.CTDH_1  AS ct ON ct.MaDH = dh.MaDH
JOIN dbo.SanPham AS sp ON sp.MaSP = ct.MaSP
WHERE dh.TrangThai IN (N'Mới', N'Đang giao');
```

Phân mảnh dọc còn đóng vai trò bảo mật. Mảnh NV_CV_1 tại chi nhánh không có cột lương, nên câu lệnh SELECT MaNV, Luong FROM NV_CV_1 báo lỗi *Invalid column name 'Luong'*. Tương tự, khi thêm vào KH_1_GD một khách hàng có MaCN = 'CN02', câu lệnh bị ràng buộc CHECK từ chối.

[ẢNH: demo_06_truy_van_cuc_bo.png – Kết quả truy vấn Q2 tại S1]

Truy vấn cục bộ tại trụ sở (S0). Báo cáo công nợ theo miền (Q3) chỉ dùng ba mảnh KH_i_TC tại S0, không cần truy cập chi nhánh nào (Bảng 4.9).

Bảng 4.9: *Báo cáo công nợ theo miền (Q3)*

Chi nhánh	Số khách	Tổng hạn mức (đồng)	Tổng công nợ (đồng)	Tỉ lệ sử dụng
CN01 – Hà Nội	10	1.700.000.000	774.500.000	45,6%
CN02 – Đà Nẵng	10	2.000.000.000	1.092.100.000	54,6%
CN03 – TP.HCM	10	1.850.000.000	967.700.000	52,3%

Truy vấn toàn cục qua view. Danh sách khách hàng có công nợ từ 80% hạn mức trở lên cần họ tên (ở chi nhánh) và công nợ (ở trụ sở), nên phải ghép mảnh của nhiều site. Truy vấn viết trên v_KhachHang như với một bảng bình thường. Kết quả gồm ba khách: KH011 (CN02, 90,1%), KH025 (CN01, 80,7%) và KH015 (CN03, 80,4%).

Báo cáo doanh thu theo chi nhánh (Q6) được viết theo hai cách cho cùng một kết quả (Bảng 4.10):

- **Cách 1**, trên view toàn cục: v_DonHang ⋈ v_KhachHang ⋈ ChiNhanh, nhóm theo chi nhánh.
- **Cách 2**, phân rã theo mảnh: mỗi site tự tính tổng trên mảnh DH_i của mình, trụ sở chỉ hợp ba dòng kết quả. Cách này không cần phép kết nối nào, vì đơn hàng đã

được phân mảnh dẫn xuất theo chi nhánh của khách. Đây chính là phép rút gọn chương trình định vị đã trình bày ở mục 3.7.2.

Bảng 4.10: *Doanh thu theo chi nhánh, không tính đơn đã hủy (Q6)*

Chi nhánh	Số đơn	Doanh thu (đồng)
CN01 – Hà Nội	14	262.670.000
CN02 – Đà Nẵng	13	256.960.000
CN03 – TP.HCM	15	171.350.000
Tổng	**42**	**690.980.000**

[ẢNH: demo_07_truy_van_toan_cuc.png – Kết quả truy vấn toàn cục qua view]

4.7.5 Giao dịch phân tán

Thiết kế cũng có cái giá phải trả. Khi lập đơn hàng, chi nhánh ghi vào DH_i và $CTDH_i$ tại S_i, nhưng đồng thời phải kiểm tra hạn mức và cộng công nợ vào KH_i_TC tại S0. Một thao tác nghiệp vụ vì vậy ghi lên hai site và phải đảm bảo tính nguyên tử. Nhóm cài đặt thủ tục usp_LapDonHang tại S1:

```
CREATE OR ALTER PROCEDURE dbo.usp_LapDonHang
    @MaDH CHAR(5), @MaKH CHAR(5), @MaNV CHAR(4), @MaSP CHAR(4), @SoLuong INT
AS
BEGIN
    SET XACT_ABORT ON;   -- lỗi bất kỳ → hủy toàn bộ giao dịch trên cả hai site
    DECLARE @DonGia DECIMAL(18,0) = (SELECT GiaBan FROM dbo.SanPham WHERE MaSP = @MaSP);
    DECLARE @Tien   DECIMAL(18,0) = @DonGia * @SoLuong;
    BEGIN TRANSACTION;
        DECLARE @HanMuc DECIMAL(18,0), @CongNo DECIMAL(18,0);
        SELECT @HanMuc = HanMucNo, @CongNo = CongNo
        FROM QLDH_TRUSO.dbo.KH_1_TC WITH (UPDLOCK, ROWLOCK)     -- đọc tại S0
        WHERE MaKH = @MaKH;
        IF @CongNo + @Tien > @HanMuc
            THROW 50002, N'Đơn hàng vượt hạn mức công nợ.', 1;
        INSERT INTO dbo.DH_1 (...) VALUES (...);                -- ghi tại S1
        INSERT INTO dbo.CTDH_1 (...) VALUES (...);
        UPDATE QLDH_TRUSO.dbo.KH_1_TC                            -- ghi tại S0
        SET CongNo = CongNo + @Tien WHERE MaKH = @MaKH;
    COMMIT TRANSACTION;
END
```

Hai trường hợp được thử nghiệm:

- **Thành công:** khách KH007 (hạn mức 200.000.000 đồng, công nợ 86.100.000 đồng) mua một máy in giá 4.590.000 đồng. Đơn DH046 xuất hiện ở S1, đồng thời công nợ tại S0 tăng lên 90.690.000 đồng. Đơn mới cũng thấy được ngay qua view toàn cục v_DonHang.
- **Thất bại:** khách KH022 (hạn mức 50.000.000 đồng, công nợ 30.900.000 đồng) mua hai laptop, tổng 33.980.000 đồng, vượt hạn mức. Thủ tục báo lỗi 50002 và giao dịch bị hủy trên **cả hai** site: không có đơn DH047 ở S1, công nợ tại S0 giữ nguyên 30.900.000 đồng.

Trên cùng một instance, SQL Server tự đảm bảo tính nguyên tử cho giao dịch trên nhiều CSDL. Khi các site là các máy khác nhau nối qua Linked Server, giao dịch được điều phối bởi MSDTC theo giao thức cam kết hai pha (2PC).

[ẢNH: demo_08_giao_dich.png – Công nợ trước và sau giao dịch, thông báo lỗi khi vượt hạn mức]

4.8 Đánh giá

Kết quả đạt được:

- Các ứng dụng có tần suất cao (Q1, Q2, Q4) chạy hoàn toàn cục bộ tại chi nhánh. Các ứng dụng của trụ sở (Q3, Q5, Q7) chạy cục bộ tại S0. Chỉ báo cáo toàn công ty (Q6) cần truy cập nhiều site, và nhờ phân mảnh dẫn xuất, nó được phân rã thành các phép tính cục bộ cộng một phép hợp nhỏ.
- Dữ liệu nhân sự nhạy cảm chỉ tồn tại ở trụ sở; chi nhánh không thể truy cập kể cả khi cố ý.
- Vị từ phân mảnh được ràng buộc bằng CHECK, nên dữ liệu không thể bị đặt sai mảnh.
- Tính đúng đắn được chứng minh về lý thuyết và kiểm tra tự động trên dữ liệu.
- Người dùng ở trụ sở có trong suốt phân mảnh và trong suốt vị trí nhờ view và synonym.

Hạn chế và đánh đổi:

- Lập đơn hàng trở thành giao dịch phân tán vì công nợ nằm ở trụ sở. Nếu đường truyền đến trụ sở gián đoạn, chi nhánh không lập được đơn. Có thể cân nhắc để công nợ tại chi nhánh và định kỳ đồng bộ về trụ sở, nhưng khi đó phải chấp nhận dữ liệu báo cáo có độ trễ.
- Khách hàng chuyển chi nhánh đòi hỏi di chuyển bộ dữ liệu giữa các mảnh ở cả hai nhánh dọc và các đơn hàng dẫn xuất.
- Bản nhân bản SanPham phải được đồng bộ mỗi khi trụ sở đổi giá.
- Thực nghiệm mô phỏng các site bằng nhiều CSDL trên một máy, chưa đo được độ trễ mạng thực tế.

4.9 Kết chương

Chương 4 đã áp dụng phân mảnh hỗn hợp cho hệ thống quản lý đơn hàng: phân tích ứng dụng, thiết kế phân mảnh HV cho KhachHang, VH cho NhanVien, phân mảnh dẫn xuất cho DonHang và ChiTietDonHang, nhân bản ChiNhanh và SanPham, lập lược đồ cấp phát, và cài đặt trên SQL Server kèm kiểm tra tính đúng đắn, truy vấn và giao

dịch phân tán. Kết quả cho thấy thiết kế giúp đa số truy vấn chạy cục bộ, đổi lại một số thao tác cập nhật trở thành giao dịch phân tán.

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 Kết quả đạt được

Về mặt lý thuyết, đồ án đã hệ thống hóa các kiến thức nền tảng về cơ sở dữ liệu phân tán: khái niệm, kiến trúc tham chiếu, các mức trong suốt và hai bài toán thiết kế là phân mảnh và cấp phát. Đồ án trình bày phân mảnh ngang nguyên thủy với thuật toán COM_MIN, phân mảnh ngang dẫn xuất và phân mảnh dọc với các thuật toán BEA và SPLIT, có ví dụ tính toán cụ thể. Trọng tâm là phân mảnh hỗn hợp: định nghĩa, hai chiến lược HV và VH cùng điều kiện để chúng tương đương, cây phân mảnh, công thức tái thiết, chứng minh tính đúng đắn, và ảnh hưởng của phân mảnh hỗn hợp đến cấp phát, xử lý truy vấn và giao dịch.

Về mặt ứng dụng, nhóm đã thiết kế CSDL phân tán cho hệ thống quản lý đơn hàng của một công ty có ba chi nhánh và một trụ sở: KhachHang theo chiến lược HV, NhanVien theo chiến lược VH, DonHang và ChiTietDonHang theo phân mảnh dẫn xuất, ChiNhanh và SanPham được nhân bản toàn phần. Thiết kế được cài đặt trên SQL Server với các view tái thiết đảm bảo trong suốt phân mảnh và vị trí, bộ truy vấn kiểm tra tự động ba quy tắc đúng đắn, các truy vấn minh họa và một giao dịch phân tán có kiểm tra hạn mức.

Qua đề tài, nhóm rút ra rằng phân mảnh hỗn hợp phát huy hiệu quả khi các nhóm người dùng khác nhau cần những phần dữ liệu khác nhau **cả theo dòng lẫn theo cột**. Lựa chọn giữa HV và VH phụ thuộc vào tiêu chí nào phân biệt các ứng dụng rõ hơn và phần dữ liệu nào không cần chia tiếp. Thiết kế phân mảnh luôn là sự đánh đổi giữa tính cục bộ của truy vấn và chi phí của cập nhật phân tán.

5.2 Hạn chế

- Thực nghiệm mô phỏng các site bằng nhiều CSDL trên cùng một máy chủ, chưa phản ánh độ trễ và sự cố mạng thực tế.
- Tần suất truy cập của các ứng dụng là giả định, chưa thu thập từ hệ thống thật.
- Chưa cài đặt cơ chế đồng bộ tự động cho các bản nhân bản, cũng như quy trình di chuyển dữ liệu khi khách hàng chuyển chi nhánh.
- Chưa đo đạc và so sánh định lượng hiệu năng giữa các phương án phân mảnh.

5.3 Hướng phát triển

- Triển khai trên nhiều máy thật nối bằng Linked Server, sử dụng MSDTC cho giao dịch phân tán, và đo thời gian phản hồi của các truy vấn cục bộ và toàn cục.

- Dùng SQL Server Replication để đồng bộ tự động các bản nhân bản ChiNhanh và SanPham.
- Viết thủ tục chuyển khách hàng giữa các chi nhánh, đảm bảo di chuyển đồng bộ các mảnh GD, TC và đơn hàng dẫn xuất trong một giao dịch.
- So sánh định lượng các phương án: phân mảnh ngang thuần, phân mảnh dọc thuần và phân mảnh hỗn hợp, trên cùng tập ứng dụng và dữ liệu lớn hơn.
- Tìm hiểu cách các hệ thống phân tán hiện đại như Citus hay CockroachDB phân mảnh và cấp phát dữ liệu tự động.

TÀI LIỆU THAM KHẢO

[1] M. T. Özsu and P. Valduriez, *Principles of Distributed Database Systems*, 4th ed. Cham, Switzerland: Springer, 2020.

[2] S. Ceri and G. Pelagatti, *Distributed Databases: Principles and Systems*. New York, NY, USA: McGraw-Hill, 1984.

[3] Microsoft, "CREATE VIEW (Transact-SQL) – Partitioned views," Microsoft Learn. [Online]. Available: https://learn.microsoft.com/en-us/sql/t-sql/statements/create-view-transact-sql

[4] W. T. McCormick, P. J. Schweitzer, and T. W. White, "Problem decomposition and data reorganization by a clustering technique," *Operations Research*, vol. 20, no. 5, pp. 993–1009, 1972.

[5] S. Navathe, S. Ceri, G. Wiederhold, and J. Dou, "Vertical partitioning algorithms for database design," *ACM Transactions on Database Systems*, vol. 9, no. 4, pp. 680–710, 1984.

[6] Microsoft, "Linked servers (Database Engine)," Microsoft Learn. [Online]. Available: https://learn.microsoft.com/en-us/sql/relational-databases/linked-servers/linked-servers-database-engine

[7] Microsoft, "Synonyms (Database Engine)," Microsoft Learn. [Online]. Available: https://learn.microsoft.com/en-us/sql/relational-databases/synonyms/synonyms-database-engine

PHỤ LỤC

Toàn bộ mã nguồn thực nghiệm nằm trong thư mục demo/ nộp kèm báo cáo: script 01_global_schema.sql (lược đồ toàn cục và dữ liệu mẫu), 02_fragments.sql (tạo site và các mảnh), chương trình sinh dữ liệu gen_demo_sql.py và hướng dẫn chạy README.md. Phụ lục này trích hai script chính dùng để tái thiết, kiểm tra và truy vấn.

Phụ lục A. Script tái thiết và kiểm tra tính đúng đắn

```
/* ====================================================================
   03_reconstruct_check.sql
   Chạy tại S0 (QLDH_TRUSO), SAU 02_fragments.sql.
   1. Tạo SYNONYM trỏ tới các mảnh nằm ở site chi nhánh
      → muốn chạy phân tán thật thì chỉ cần đổi synonym sang Linked Server.
   2. Tạo VIEW tái thiết quan hệ toàn cục (trong suốt phân mảnh + vị trí).
   3. Kiểm tra 3 quy tắc đúng đắn: ĐẦY ĐỦ – TÁI THIẾT – RỜI NHAU.
      Mong đợi: mọi dòng kết quả có SoDongLech = 0, KetQua = ĐẠT.
   ==================================================================== */
USE QLDH_TRUSO;
GO

/* --------------------------------------------------------------------
   1. SYNONYM: tên cục bộ ở S0 → mảnh ở site khác
   -------------------------------------------------------------------- */
-- DROP SYNONYM chỉ nhận một tên mỗi lệnh
DROP SYNONYM IF EXISTS dbo.syn_KH_1_GD;
DROP SYNONYM IF EXISTS dbo.syn_NV_CV_1;
DROP SYNONYM IF EXISTS dbo.syn_DH_1;
DROP SYNONYM IF EXISTS dbo.syn_CTDH_1;
DROP SYNONYM IF EXISTS dbo.syn_KH_2_GD;
DROP SYNONYM IF EXISTS dbo.syn_NV_CV_2;
DROP SYNONYM IF EXISTS dbo.syn_DH_2;
DROP SYNONYM IF EXISTS dbo.syn_CTDH_2;
DROP SYNONYM IF EXISTS dbo.syn_KH_3_GD;
DROP SYNONYM IF EXISTS dbo.syn_NV_CV_3;
DROP SYNONYM IF EXISTS dbo.syn_DH_3;
DROP SYNONYM IF EXISTS dbo.syn_CTDH_3;

-- S1 – Hà Nội
CREATE SYNONYM dbo.syn_KH_1_GD FOR QLDH_HN.dbo.KH_1_GD;
CREATE SYNONYM dbo.syn_NV_CV_1 FOR QLDH_HN.dbo.NV_CV_1;
CREATE SYNONYM dbo.syn_DH_1    FOR QLDH_HN.dbo.DH_1;
CREATE SYNONYM dbo.syn_CTDH_1  FOR QLDH_HN.dbo.CTDH_1;
-- S2 – Đà Nẵng
CREATE SYNONYM dbo.syn_KH_2_GD FOR QLDH_DN.dbo.KH_2_GD;
CREATE SYNONYM dbo.syn_NV_CV_2 FOR QLDH_DN.dbo.NV_CV_2;
CREATE SYNONYM dbo.syn_DH_2    FOR QLDH_DN.dbo.DH_2;
CREATE SYNONYM dbo.syn_CTDH_2  FOR QLDH_DN.dbo.CTDH_2;
-- S3 – TP.HCM
CREATE SYNONYM dbo.syn_KH_3_GD FOR QLDH_HCM.dbo.KH_3_GD;
CREATE SYNONYM dbo.syn_NV_CV_3 FOR QLDH_HCM.dbo.NV_CV_3;
CREATE SYNONYM dbo.syn_DH_3    FOR QLDH_HCM.dbo.DH_3;
CREATE SYNONYM dbo.syn_CTDH_3  FOR QLDH_HCM.dbo.CTDH_3;
GO

/* --------------------------------------------------------------------
   2. VIEW TÁI THIẾT
   UNION ALL (không phải UNION) là đúng vì các mảnh ngang RỜI NHAU.
   -------------------------------------------------------------------- */

-- KhachHang = ∪ i=1..3 ( KH_i_GD ⋈[MaKH] KH_i_TC )            (hỗn hợp HV)
CREATE OR ALTER VIEW dbo.v_KhachHang AS
    SELECT gd.MaKH, gd.HoTen, gd.SDT, gd.Email, gd.DiaChi, gd.MaCN,
           tc.HanMucNo, tc.CongNo, tc.DiemTichLuy
```

```
    FROM dbo.syn_KH_1_GD AS gd JOIN dbo.KH_1_TC AS tc ON tc.MaKH = gd.MaKH
    UNION ALL
    SELECT gd.MaKH, gd.HoTen, gd.SDT, gd.Email, gd.DiaChi, gd.MaCN,
           tc.HanMucNo, tc.CongNo, tc.DiemTichLuy
    FROM dbo.syn_KH_2_GD AS gd JOIN dbo.KH_2_TC AS tc ON tc.MaKH = gd.MaKH
    UNION ALL
    SELECT gd.MaKH, gd.HoTen, gd.SDT, gd.Email, gd.DiaChi, gd.MaCN,
           tc.HanMucNo, tc.CongNo, tc.DiemTichLuy
    FROM dbo.syn_KH_3_GD AS gd JOIN dbo.KH_3_TC AS tc ON tc.MaKH = gd.MaKH;
GO

-- NhanVien = ( NV_CV_1 ∪ NV_CV_2 ∪ NV_CV_3 ) ⋈[MaNV] NV_NS      (hỗn hợp VH)
CREATE OR ALTER VIEW dbo.v_NhanVien AS
    SELECT cv.MaNV, cv.HoTen, cv.ChucVu, cv.SDT, cv.MaCN,
           ns.NgaySinh, ns.CCCD, ns.Luong, ns.SoTaiKhoan
    FROM (
        SELECT MaNV, HoTen, ChucVu, SDT, MaCN FROM dbo.syn_NV_CV_1
        UNION ALL
        SELECT MaNV, HoTen, ChucVu, SDT, MaCN FROM dbo.syn_NV_CV_2
        UNION ALL
        SELECT MaNV, HoTen, ChucVu, SDT, MaCN FROM dbo.syn_NV_CV_3
    ) AS cv
    JOIN dbo.NV_NS AS ns ON ns.MaNV = cv.MaNV;
GO

-- DonHang = DH_1 ∪ DH_2 ∪ DH_3                                    (ngang dẫn xuất)
CREATE OR ALTER VIEW dbo.v_DonHang AS
    SELECT MaDH, NgayDat, MaKH, MaNV, TrangThai, TongTien FROM dbo.syn_DH_1
    UNION ALL
    SELECT MaDH, NgayDat, MaKH, MaNV, TrangThai, TongTien FROM dbo.syn_DH_2
    UNION ALL
    SELECT MaDH, NgayDat, MaKH, MaNV, TrangThai, TongTien FROM dbo.syn_DH_3;
GO

-- ChiTietDonHang = CTDH_1 ∪ CTDH_2 ∪ CTDH_3                      (ngang dẫn xuất)
CREATE OR ALTER VIEW dbo.v_ChiTietDonHang AS
    SELECT MaDH, MaSP, SoLuong, DonGia FROM dbo.syn_CTDH_1
    UNION ALL
    SELECT MaDH, MaSP, SoLuong, DonGia FROM dbo.syn_CTDH_2
    UNION ALL
    SELECT MaDH, MaSP, SoLuong, DonGia FROM dbo.syn_CTDH_3;
GO

-- Xem thử: người dùng truy vấn như một bảng duy nhất, không cần biết dữ liệu ở site nào
SELECT TOP (6) * FROM dbo.v_KhachHang ORDER BY MaKH;
SELECT TOP (6) * FROM dbo.v_NhanVien  ORDER BY MaNV;
GO

/* ---------------------------------------------------------------------
   3. KIỂM TRA TÍNH ĐÚNG ĐẮN
   So sánh với lược đồ gốc QLDH_GOC. SoDongLech = số bộ/thuộc tính vi phạm.
   --------------------------------------------------------------------- */
DECLARE @KQ TABLE (
    STT        INT IDENTITY(1,1),
    QuanHe     NVARCHAR(20),
    QuyTac     NVARCHAR(12),
    MoTa       NVARCHAR(150),
    SoDongLech INT
);

/* ===== KhachHang (hỗn hợp HV) ===== */
-- Đầy đủ (ngang): bộ của KhachHang gốc không thuộc mảnh ngang nào
INSERT INTO @KQ
SELECT N'KhachHang', N'Đầy đủ', N'Bộ không thuộc mảnh ngang nào', COUNT(*)
FROM QLDH_GOC.dbo.KhachHang AS k
WHERE NOT EXISTS (SELECT 1 FROM (SELECT MaKH FROM dbo.syn_KH_1_GD
                                 UNION ALL SELECT MaKH FROM dbo.syn_KH_2_GD
                                 UNION ALL SELECT MaKH FROM dbo.syn_KH_3_GD) AS m
                  WHERE m.MaKH = k.MaKH);
```

```
-- Đầy đủ (dọc): thuộc tính của KhachHang không nằm trong mảnh dọc nào
-- (kiểm tra trên KH_1_GD / KH_1_TC; các mảnh 2, 3 có cùng cấu trúc)
INSERT INTO @KQ
SELECT N'KhachHang', N'Đầy đủ', N'Thuộc tính không thuộc mảnh dọc nào (GD ∪ TC)', COUNT(*)
FROM (
    SELECT name FROM QLDH_GOC.sys.columns WHERE object_id =
OBJECT_ID(N'QLDH_GOC.dbo.KhachHang')
    EXCEPT
    SELECT name FROM (
        SELECT name FROM QLDH_HN.sys.columns WHERE object_id =
OBJECT_ID(N'QLDH_HN.dbo.KH_1_GD')
        UNION
        SELECT name FROM sys.columns WHERE object_id = OBJECT_ID(N'dbo.KH_1_TC')
    ) AS u
) AS x;

-- Tái thiết: số bộ lệch giữa KhachHang gốc và v_KhachHang (hai chiều + so số lượng)
INSERT INTO @KQ
SELECT N'KhachHang', N'Tái thiết', N'Bộ lệch giữa KhachHang gốc và v_KhachHang',
    (SELECT COUNT(*) FROM (
        SELECT MaKH, HoTen, SDT, Email, DiaChi, MaCN, HanMucNo, CongNo, DiemTichLuy FROM
QLDH_GOC.dbo.KhachHang
        EXCEPT
        SELECT MaKH, HoTen, SDT, Email, DiaChi, MaCN, HanMucNo, CongNo, DiemTichLuy FROM
dbo.v_KhachHang) AS a)
  + (SELECT COUNT(*) FROM (
        SELECT MaKH, HoTen, SDT, Email, DiaChi, MaCN, HanMucNo, CongNo, DiemTichLuy FROM
dbo.v_KhachHang
        EXCEPT
        SELECT MaKH, HoTen, SDT, Email, DiaChi, MaCN, HanMucNo, CongNo, DiemTichLuy FROM
QLDH_GOC.dbo.KhachHang) AS b)
  + ABS((SELECT COUNT(*) FROM dbo.v_KhachHang) - (SELECT COUNT(*) FROM
QLDH_GOC.dbo.KhachHang));

-- Rời nhau (ngang): MaKH xuất hiện ở hơn một mảnh ngang
INSERT INTO @KQ
SELECT N'KhachHang', N'Rời nhau', N'MaKH thuộc nhiều hơn 1 mảnh ngang', COUNT(*)
FROM (SELECT MaKH FROM (SELECT MaKH FROM dbo.syn_KH_1_GD
                        UNION ALL SELECT MaKH FROM dbo.syn_KH_2_GD
                        UNION ALL SELECT MaKH FROM dbo.syn_KH_3_GD) AS m
      GROUP BY MaKH HAVING COUNT(*) > 1) AS x;

-- Rời nhau (dọc): thuộc tính KHÔNG phải khóa có mặt ở cả GD và TC
INSERT INTO @KQ
SELECT N'KhachHang', N'Rời nhau', N'Thuộc tính không khóa lặp ở cả GD và TC', COUNT(*)
FROM (
    SELECT name FROM QLDH_HN.sys.columns WHERE object_id = OBJECT_ID(N'QLDH_HN.dbo.KH_1_GD')
    INTERSECT
    SELECT name FROM sys.columns WHERE object_id = OBJECT_ID(N'dbo.KH_1_TC')
) AS x
WHERE name <> N'MaKH';

/* ===== NhanVien (hỗn hợp VH) ===== */
INSERT INTO @KQ
SELECT N'NhanVien', N'Đầy đủ', N'Thuộc tính không thuộc mảnh dọc nào (CV ∪ NS)', COUNT(*)
FROM (
    SELECT name FROM QLDH_GOC.sys.columns WHERE object_id =
OBJECT_ID(N'QLDH_GOC.dbo.NhanVien')
    EXCEPT
    SELECT name FROM (
        SELECT name FROM QLDH_HN.sys.columns WHERE object_id =
OBJECT_ID(N'QLDH_HN.dbo.NV_CV_1')
        UNION
        SELECT name FROM sys.columns WHERE object_id = OBJECT_ID(N'dbo.NV_NS')
    ) AS u
) AS x;

INSERT INTO @KQ
SELECT N'NhanVien', N'Đầy đủ', N'Bộ của NV_CV không thuộc mảnh ngang nào', COUNT(*)
```

```
FROM QLDH_GOC.dbo.NhanVien AS v
WHERE NOT EXISTS (SELECT 1 FROM (SELECT MaNV FROM dbo.syn_NV_CV_1
                                 UNION ALL SELECT MaNV FROM dbo.syn_NV_CV_2
                                 UNION ALL SELECT MaNV FROM dbo.syn_NV_CV_3) AS m
                  WHERE m.MaNV = v.MaNV);

INSERT INTO @KQ
SELECT N'NhanVien', N'Tái thiết', N'Bộ lệch giữa NhanVien gốc và v_NhanVien',
    (SELECT COUNT(*) FROM (
        SELECT MaNV, HoTen, ChucVu, SDT, MaCN, NgaySinh, CCCD, Luong, SoTaiKhoan FROM
QLDH_GOC.dbo.NhanVien
        EXCEPT
        SELECT MaNV, HoTen, ChucVu, SDT, MaCN, NgaySinh, CCCD, Luong, SoTaiKhoan FROM
dbo.v_NhanVien) AS a)
  + (SELECT COUNT(*) FROM (
        SELECT MaNV, HoTen, ChucVu, SDT, MaCN, NgaySinh, CCCD, Luong, SoTaiKhoan FROM
dbo.v_NhanVien
        EXCEPT
        SELECT MaNV, HoTen, ChucVu, SDT, MaCN, NgaySinh, CCCD, Luong, SoTaiKhoan FROM
QLDH_GOC.dbo.NhanVien) AS b)
  + ABS((SELECT COUNT(*) FROM dbo.v_NhanVien) - (SELECT COUNT(*) FROM QLDH_GOC.dbo.NhanVien));

INSERT INTO @KQ
SELECT N'NhanVien', N'Rời nhau', N'MaNV thuộc nhiều hơn 1 mảnh ngang', COUNT(*)
FROM (SELECT MaNV FROM (SELECT MaNV FROM dbo.syn_NV_CV_1
                        UNION ALL SELECT MaNV FROM dbo.syn_NV_CV_2
                        UNION ALL SELECT MaNV FROM dbo.syn_NV_CV_3) AS m
      GROUP BY MaNV HAVING COUNT(*) > 1) AS x;

INSERT INTO @KQ
SELECT N'NhanVien', N'Rời nhau', N'Thuộc tính không khóa lặp ở cả CV và NS', COUNT(*)
FROM (
    SELECT name FROM QLDH_HN.sys.columns WHERE object_id = OBJECT_ID(N'QLDH_HN.dbo.NV_CV_1')
    INTERSECT
    SELECT name FROM sys.columns WHERE object_id = OBJECT_ID(N'dbo.NV_NS')
) AS x
WHERE name <> N'MaNV';

/* ===== DonHang, ChiTietDonHang (ngang dẫn xuất) ===== */
INSERT INTO @KQ
SELECT N'DonHang', N'Đầy đủ', N'Đơn hàng không thuộc mảnh nào', COUNT(*)
FROM QLDH_GOC.dbo.DonHang AS d
WHERE NOT EXISTS (SELECT 1 FROM dbo.v_DonHang AS m WHERE m.MaDH = d.MaDH);

INSERT INTO @KQ
SELECT N'DonHang', N'Tái thiết', N'Bộ lệch giữa DonHang gốc và v_DonHang',
    (SELECT COUNT(*) FROM (
        SELECT MaDH, NgayDat, MaKH, MaNV, TrangThai, TongTien FROM QLDH_GOC.dbo.DonHang
        EXCEPT
        SELECT MaDH, NgayDat, MaKH, MaNV, TrangThai, TongTien FROM dbo.v_DonHang) AS a)
  + (SELECT COUNT(*) FROM (
        SELECT MaDH, NgayDat, MaKH, MaNV, TrangThai, TongTien FROM dbo.v_DonHang
        EXCEPT
        SELECT MaDH, NgayDat, MaKH, MaNV, TrangThai, TongTien FROM QLDH_GOC.dbo.DonHang) AS b)
  + ABS((SELECT COUNT(*) FROM dbo.v_DonHang) - (SELECT COUNT(*) FROM QLDH_GOC.dbo.DonHang));

INSERT INTO @KQ
SELECT N'DonHang', N'Rời nhau', N'MaDH thuộc nhiều hơn 1 mảnh', COUNT(*)
FROM (SELECT MaDH FROM dbo.v_DonHang GROUP BY MaDH HAVING COUNT(*) > 1) AS x;

INSERT INTO @KQ
SELECT N'ChiTietDonHang', N'Tái thiết', N'Bộ lệch giữa ChiTietDonHang gốc và
v_ChiTietDonHang',
    (SELECT COUNT(*) FROM (
        SELECT MaDH, MaSP, SoLuong, DonGia FROM QLDH_GOC.dbo.ChiTietDonHang
        EXCEPT
        SELECT MaDH, MaSP, SoLuong, DonGia FROM dbo.v_ChiTietDonHang) AS a)
  + (SELECT COUNT(*) FROM (
        SELECT MaDH, MaSP, SoLuong, DonGia FROM dbo.v_ChiTietDonHang
```

```
       EXCEPT
       SELECT MaDH, MaSP, SoLuong, DonGia FROM QLDH_GOC.dbo.ChiTietDonHang) AS b)
  + ABS((SELECT COUNT(*) FROM dbo.v_ChiTietDonHang) - (SELECT COUNT(*) FROM
QLDH_GOC.dbo.ChiTietDonHang));

-- Kết quả (chụp màn hình bảng này cho slide 15 và mục 4.7)
SELECT STT, QuanHe, QuyTac, MoTa, SoDongLech,
       CASE WHEN SoDongLech = 0 THEN N'ĐẠT' ELSE N'KHÔNG ĐẠT' END AS KetQua
FROM @KQ
ORDER BY STT;
-- Mong đợi: 13 dòng, tất cả SoDongLech = 0, KetQua = ĐẠT
GO
```

Phụ lục B. Script truy vấn minh họa và giao dịch phân tán

```
/* ====================================================================
   04_queries.sql
   Truy vấn minh họa theo bảng ứng dụng Q1–Q7 (docs/01_bai_toan_thiet_ke.md).
   Chạy SAU 03_reconstruct_check.sql. Chạy lại bao nhiêu lần cũng được:
   phần D (giao dịch) tự hoàn tác nên không làm lệch dữ liệu.
     A. Truy vấn CỤC BỘ tại chi nhánh Hà Nội (S1)
     B. Truy vấn CỤC BỘ tại trụ sở (S0)
     C. Truy vấn TOÀN CỤC qua view (trong suốt phân mảnh)
     D. Giao dịch phân tán: lập đơn ở S1, cập nhật công nợ ở S0
   ==================================================================== */

/* ====================================================================
   A. TẠI CHI NHÁNH HÀ NỘI – S1 (QLDH_HN): chỉ đọc mảnh nằm tại chỗ
   ==================================================================== */
USE QLDH_HN;
GO

-- Q1: tra cứu khách hàng của chi nhánh để lập đơn            → 10 dòng
SELECT MaKH, HoTen, SDT, Email, DiaChi
FROM dbo.KH_1_GD
ORDER BY MaKH;

-- Q2: đơn đang xử lý (Mới / Đang giao) kèm chi tiết           → 6 dòng (DH020, DH027, DH028,
DH036)
--     DonHang ⋈ KhachHang chạy cục bộ nhờ phân mảnh ngang DẪN XUẤT:
--     DH_1 và KH_1_GD cùng nằm ở S1
SELECT dh.MaDH, dh.NgayDat, kh.HoTen AS KhachHang, nv.HoTen AS NhanVien,
       sp.TenSP, ct.SoLuong, ct.DonGia, ct.SoLuong * ct.DonGia AS ThanhTien, dh.TrangThai
FROM dbo.DH_1    AS dh
JOIN dbo.KH_1_GD AS kh ON kh.MaKH = dh.MaKH
JOIN dbo.NV_CV_1 AS nv ON nv.MaNV = dh.MaNV
JOIN dbo.CTDH_1  AS ct ON ct.MaDH = dh.MaDH
JOIN dbo.SanPham AS sp ON sp.MaSP = ct.MaSP          -- SanPham là bản nhân bản tại S1
WHERE dh.TrangThai IN (N'Mới', N'Đang giao')
ORDER BY dh.NgayDat, dh.MaDH;

-- Q4: danh sách nhân viên chi nhánh                          → 4 dòng
SELECT MaNV, HoTen, ChucVu, SDT FROM dbo.NV_CV_1 ORDER BY MaNV;

-- Bảo mật nhờ phân mảnh dọc: chi nhánh KHÔNG có cột lương.
-- Bỏ comment dòng dưới sẽ báo lỗi: Invalid column name 'Luong'.
-- SELECT MaNV, Luong FROM dbo.NV_CV_1;

-- Vị từ phân mảnh được ràng buộc bằng CHECK: mảnh Hà Nội không nhận khách CN02.
-- Bỏ comment sẽ báo lỗi: The INSERT statement conflicted with the CHECK constraint
"CK_KH_1_GD_MaCN".
-- INSERT INTO dbo.KH_1_GD (MaKH, HoTen, SDT, Email, DiaChi, MaCN)
-- VALUES ('KH999', N'Khách thử', '0900000000', NULL, N'Đà Nẵng', 'CN02');
GO

/* ====================================================================
   B. TẠI TRỤ SỞ – S0 (QLDH_TRUSO): chỉ đọc mảnh nằm tại chỗ
   ==================================================================== */
```

```
USE QLDH_TRUSO;
GO

-- Q3: báo cáo công nợ theo miền, chỉ dùng các mảnh KH_i_TC ở S0.
--     Mảnh TC không có cột MaCN; miền được xác định bởi chính mảnh (σ ở bước 1 của HV).
--     Mong đợi: CN01 10 KH, hạn mức 1.700.000.000, công nợ   774.500.000, 45.6%
--               CN02 10 KH, hạn mức 2.000.000.000, công nợ 1.092.100.000, 54.6%
--               CN03 10 KH, hạn mức 1.850.000.000, công nợ   967.700.000, 52.3%
SELECT cn.MaCN, cn.TenCN, x.SoKhach, x.TongHanMuc, x.TongCongNo,
       CAST(100.0 * x.TongCongNo / x.TongHanMuc AS DECIMAL(5,1)) AS TyLeSuDung
FROM (
    SELECT 'CN01' AS MaCN, COUNT(*) AS SoKhach, SUM(HanMucNo) AS TongHanMuc, SUM(CongNo) AS
TongCongNo FROM dbo.KH_1_TC
    UNION ALL
    SELECT 'CN02', COUNT(*), SUM(HanMucNo), SUM(CongNo) FROM dbo.KH_2_TC
    UNION ALL
    SELECT 'CN03', COUNT(*), SUM(HanMucNo), SUM(CongNo) FROM dbo.KH_3_TC
) AS x
JOIN dbo.ChiNhanh AS cn ON cn.MaCN = x.MaCN          -- ChiNhanh là bản nhân bản tại S0
ORDER BY cn.MaCN;

-- Q5: bảng lương – dữ liệu nhạy cảm chỉ có ở S0               → 12 dòng
SELECT MaNV, CCCD, Luong, SoTaiKhoan FROM dbo.NV_NS ORDER BY MaNV;
GO

/* ======================================================================
   C. TOÀN CỤC QUA VIEW (S0): người dùng không cần biết mảnh nằm ở đâu
   ====================================================================== */

-- Q3b: khách sắp vượt hạn mức (công nợ >= 80%). Cần HoTen (ở chi nhánh) + CongNo (ở S0),
--      nên phải ghép mảnh của nhiều site → truy vấn phân tán.
--      Mong đợi 3 dòng: KH011 Bùi Bảo Lan (CN02) 90.1% · KH025 Hoàng Bảo Yến (CN01) 80.7%
--                       · KH015 Ngô Minh Ngân (CN03) 80.4%
SELECT MaKH, HoTen, MaCN, HanMucNo, CongNo,
       CAST(100.0 * CongNo / HanMucNo AS DECIMAL(5,1)) AS TyLe
FROM dbo.v_KhachHang
WHERE CongNo >= 0.8 * HanMucNo
ORDER BY 1.0 * CongNo / HanMucNo DESC;

-- Q6a: doanh thu theo chi nhánh (không tính đơn đã hủy), viết trên view toàn cục.
--      Mong đợi: CN01 14 đơn 262.670.000 · CN02 13 đơn 256.960.000 · CN03 15 đơn 171.350.000
SELECT kh.MaCN, cn.TenCN, COUNT(*) AS SoDon, SUM(dh.TongTien) AS DoanhThu
FROM dbo.v_DonHang   AS dh
JOIN dbo.v_KhachHang AS kh ON kh.MaKH = dh.MaKH
JOIN dbo.ChiNhanh    AS cn ON cn.MaCN = kh.MaCN
WHERE dh.TrangThai <> N'Đã hủy'
GROUP BY kh.MaCN, cn.TenCN
ORDER BY kh.MaCN;

-- Q6b: cùng kết quả, nhưng viết theo cách phân rã truy vấn: mỗi site tự tính trên mảnh
--      của mình, S0 chỉ gộp 3 dòng. Không cần phép kết nối nào, vì đơn hàng đã được
--      phân mảnh dẫn xuất theo chi nhánh của khách.
SELECT 'CN01' AS MaCN, COUNT(*) AS SoDon, SUM(TongTien) AS DoanhThu FROM dbo.syn_DH_1 WHERE
TrangThai <> N'Đã hủy'
UNION ALL
SELECT 'CN02', COUNT(*), SUM(TongTien) FROM dbo.syn_DH_2 WHERE TrangThai <> N'Đã hủy'
UNION ALL
SELECT 'CN03', COUNT(*), SUM(TongTien) FROM dbo.syn_DH_3 WHERE TrangThai <> N'Đã hủy';

-- Q6c: doanh thu toàn công ty theo tháng
--      Mong đợi: 2026-09 22 đơn 279.000.000 · 2026-10 20 đơn 411.980.000
SELECT YEAR(NgayDat) AS Nam, MONTH(NgayDat) AS Thang, COUNT(*) AS SoDon, SUM(TongTien) AS
DoanhThu
FROM dbo.v_DonHang
WHERE TrangThai <> N'Đã hủy'
GROUP BY YEAR(NgayDat), MONTH(NgayDat)
ORDER BY Nam, Thang;

-- Q6d: top 3 sản phẩm theo doanh thu
```

```
--      Mong đợi: SP01 (390.770.000) · SP05 (110.160.000) · SP02 (69.090.000)
SELECT TOP (3) sp.MaSP, sp.TenSP, SUM(ct.SoLuong) AS SoLuong, SUM(ct.SoLuong * ct.DonGia) AS
DoanhThu
FROM dbo.v_ChiTietDonHang AS ct
JOIN dbo.v_DonHang        AS dh ON dh.MaDH = ct.MaDH
JOIN dbo.SanPham          AS sp ON sp.MaSP = ct.MaSP
WHERE dh.TrangThai <> N'Đã hủy'
GROUP BY sp.MaSP, sp.TenSP
ORDER BY DoanhThu DESC;


-- Tổng quỹ lương theo chi nhánh: ghép NV_CV (ở chi nhánh) với NV_NS (ở S0)
--      Mong đợi: CN01 61.000.000 · CN02 62.000.000 · CN03 62.000.000
SELECT MaCN, COUNT(*) AS SoNhanVien, SUM(Luong) AS QuyLuong
FROM dbo.v_NhanVien
GROUP BY MaCN
ORDER BY MaCN;
GO


/* ====================================================================
   D. GIAO DỊCH PHÂN TÁN – cái giá của thiết kế
   Lập đơn ở S1 (DH_1, CTDH_1), nhưng công nợ nằm ở S0 (KH_1_TC).
   Một thao tác nghiệp vụ phải ghi lên 2 site và cần tính nguyên tử: cả hai
   cùng thành công hoặc cùng hủy.
   (Cùng instance thì SQL Server tự bảo đảm. Qua Linked Server cần MSDTC
    và BEGIN DISTRIBUTED TRANSACTION, tức giao thức cam kết 2 pha 2PC.)
   ==================================================================== */
USE QLDH_HN;
GO
CREATE OR ALTER PROCEDURE dbo.usp_LapDonHang
    @MaDH CHAR(5), @MaKH CHAR(5), @MaNV CHAR(4), @MaSP CHAR(4), @SoLuong INT
AS
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON;   -- lỗi bất kỳ → hủy toàn bộ giao dịch trên cả 2 site

    DECLARE @DonGia DECIMAL(18,0) = (SELECT GiaBan FROM dbo.SanPham WHERE MaSP = @MaSP);
    DECLARE @Tien   DECIMAL(18,0) = @DonGia * @SoLuong;

    IF NOT EXISTS (SELECT 1 FROM dbo.KH_1_GD WHERE MaKH = @MaKH)
        THROW 50001, N'Khách hàng không thuộc chi nhánh Hà Nội.', 1;

    BEGIN TRANSACTION;
        -- Đọc hạn mức ở S0, khóa dòng để không ai cập nhật chen ngang
        DECLARE @HanMuc DECIMAL(18,0), @CongNo DECIMAL(18,0);
        SELECT @HanMuc = HanMucNo, @CongNo = CongNo
        FROM QLDH_TRUSO.dbo.KH_1_TC WITH (UPDLOCK, ROWLOCK)
        WHERE MaKH = @MaKH;

        IF @CongNo + @Tien > @HanMuc
            THROW 50002, N'Đơn hàng vượt hạn mức công nợ – giao dịch bị hủy trên cả 2 site.',
1;

        -- Ghi tại S1
        INSERT INTO dbo.DH_1 (MaDH, NgayDat, MaKH, MaNV, TrangThai, TongTien)
        VALUES (@MaDH, CAST(GETDATE() AS DATE), @MaKH, @MaNV, N'Mới', @Tien);
        INSERT INTO dbo.CTDH_1 (MaDH, MaSP, SoLuong, DonGia)
        VALUES (@MaDH, @MaSP, @SoLuong, @DonGia);

        -- Ghi tại S0
        UPDATE QLDH_TRUSO.dbo.KH_1_TC
        SET CongNo = CongNo + @Tien
        WHERE MaKH = @MaKH;
    COMMIT TRANSACTION;
END
GO

-- Trường hợp 1: THÀNH CÔNG. KH007 (hạn mức 200.000.000, công nợ 86.100.000) mua 1 máy in
4.590.000.
```

```
-- Bọc trong giao dịch ngoài rồi ROLLBACK để demo chạy lại được và không làm lệch so với
QLDH_GOC.
-- (Đổi ROLLBACK thành COMMIT nếu muốn lưu thật; khi đó 03 sẽ báo lệch vì QLDH_GOC không đổi.)
BEGIN TRANSACTION;
    SELECT N'Trước' AS Luc, MaKH, HanMucNo, CongNo FROM QLDH_TRUSO.dbo.KH_1_TC WHERE MaKH =
'KH007';

    EXEC dbo.usp_LapDonHang @MaDH = 'DH046', @MaKH = 'KH007', @MaNV = 'NV02', @MaSP = 'SP05',
@SoLuong = 1;

    SELECT N'Sau' AS Luc, MaKH, HanMucNo, CongNo FROM QLDH_TRUSO.dbo.KH_1_TC WHERE MaKH =
'KH007';   -- công nợ 90.690.000
    SELECT * FROM QLDH_TRUSO.dbo.v_DonHang WHERE MaDH = 'DH046';      -- đơn mới thấy được qua
view toàn cục
ROLLBACK TRANSACTION;
GO

-- Trường hợp 2: THẤT BẠI. KH022 (hạn mức 50.000.000, công nợ 30.900.000) mua 2 laptop
33.980.000.
BEGIN TRY
    EXEC dbo.usp_LapDonHang @MaDH = 'DH047', @MaKH = 'KH022', @MaNV = 'NV03', @MaSP = 'SP01',
@SoLuong = 2;
END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
    SELECT ERROR_NUMBER() AS MaLoi, ERROR_MESSAGE() AS ThongBao;   -- 50002
END CATCH;

-- Kiểm chứng tính nguyên tử: không site nào bị ghi dở
SELECT (SELECT COUNT(*) FROM dbo.DH_1 WHERE MaDH = 'DH047')                    AS
DonDH047_TaiS1,   -- 0
       (SELECT CongNo  FROM QLDH_TRUSO.dbo.KH_1_TC WHERE MaKH = 'KH022')       AS
CongNoKH022_TaiS0; -- 30.900.000
GO
```